第 23 课：Agent 开发面试辅导与项目简历

适用课程：Go 企业级 AI Agent 平台实战课程 ，欢迎来B站关注 @七米老师 、@七米课堂
面试题来源：公开面经检索 + 群友真实面试题分享 + AI 扩展
代码依据：kbot 项目代码。

全文分为两部分：

- 第一部分为公开面经题目、学员真实面试题、课程化参考答案及延伸追问；
- 第二部分为将 Kbot 项目基于海外电商、保险业务场景编写两份可用于简历的项目介绍与说明。

一、面试真题

1.1 收集范围与使用方法

围绕 Agent 开发、大模型应用、AI 全栈和相关后端岗位检索公开面经，筛选出有价值的一些个人面试记录，覆盖百度、字节、阿里、美团、快手、京东、4399、vivo、恒生电子、迅雷、经纬恒润、腾讯等企业；另收录课程学员实际面试反馈的 7 道题。目前共 116 道带答案的练习题，其中 71 道为公开面经题，38 道为课程延伸题，7 道为学员实面题。公开记录包含明确的 Agent 岗位及相关应用、全栈、后端岗位，范围在来源表中区分。

- **公开面经题**：对应作者公开回忆的问题，提供原帖链接；题干经过概括，保留考察意图。企业、岗位及面试时间均为作者自述，未得到企业独立确认。
- **学员实面题**：由课程方提供学员实际面试时遇到的问题，本次反馈编号 U01。题目记录与讲解见学员实面专项。
- **课程延伸题**：根据项目源码、协议文档和面经考点设计，用于应对面试时的深入追问。
- **参考答案**：以已完成相应课程实验的面试者为视角，采用可口述的第一人称回答，展开原理、判断依据、处理流程、项目案例与验证边界。
- **实现边界**：默认以最终版为回答依据；标注“扩展方案”的内容需要学员完成实现、测试后再写入个人成果。

先用 Q81–Q87 演练学员实面问题，再复习 Q02、Q04、Q06、Q20、Q26、Q30、Q32、Q37、Q39、Q49、Q51、Q55、Q59、Q63、Q71–Q74，按目标岗位补齐其余题目。

练习时先掌握完整回答，再根据面试官追问调整展开深度。分类题先说有哪些情况，再说如何识别，最后分别解释处理方法；方案题讲清前提、选择理由、执行步骤和失败分支；项目题用实际请求、代码与验证记录支撑结论。Q49 提供了“中断分类—识别信号—分情况恢复—副作用核对”的完整示例。

1.2 kbot 项目的实际能力

能力	课程与源码依据	面试表达边界
Agent 主循环	第 05、09 课；Eino ADK `ChatModelAgent + Runner + ToolsNode`	使用成熟 ADK，平台补充版本、权限、审批、审计与计量
RAG	第 10、11 课；教学/内存路径有 BM25，最终持久化路径使用 PostgreSQL FTS、pgvector、RRF	`ts_rank_cd` 属于 PostgreSQL 全文检索排序；准确说明各实现，避免统称为原生 BM25
重排	内存检索器有词重叠轻量排序	当前 PostgreSQL 路径没有独立 Cross-Encoder 重排模型
会话记忆	第 14 课；最终版加载持久化消息，取最近 10 条历史 user/assistant 消息	10 条消息与 10 轮对话含义不同；长期语义记忆、自动摘要属于扩展
Skill	第 13 课；渐进式披露、Tool/KB 权限收窄	Skill 描述流程，执行授权由服务端落实
审批与恢复	第 16、17 课；StatefulInterrupt、checkpoint、精确 interrupt address、ResumeWithParams	审批恢复链路有实现；任意步骤崩溃后的通用任务恢复需要额外设计
多 Agent	Supervisor、AgentTool 与顺序 Pipeline	当前 Supervisor 配置顺序执行；大规模并行 DAG 调度属于扩展
MCP	Streamable HTTP；stdio 适配与测试保留	App/Worker 控制面拒绝直接配置 stdio；生产接入需独立 Runner
A2A	固定协议版本、AgentCard、JSON-RPC `SendMessage`	完整任务订阅、推送、AgentCard 签名验证需要继续实现
业务集成	两个独立 Go Module 的业务模拟器	使用模拟 OMS/WMS/结算/保险数据，真实业务系统对接需要单独交付
业务可靠性	平台有持久化和审批租约；业务模拟器以进程内 map 保存状态与幂等结果	业务幂等记录的持久化、参数冲突校验及真实资金通道仍需补齐
评测	Eval、确定性 Judge、LLM Judge、发布门禁；跨境 3 条、保险 4 条业务样本	小样本和 Mock E2E 用于验收流程，不能据此宣称线上准确率或业务收益

完整映射见课堂与最终版能力对照、最终版架构和业务资产预设。四个预置场景 Agent 与独立项目安装的综合 Agent 使用不同配置；演示前应确认当前 Workspace、Agent 名称及绑定资产。

1.3 Agent 架构、范式与框架选择（Q01–Q10）

代码入口：Runtime Engine、ADK 装配、Supervisor、Pipeline。对应第 01、09、14、21 课及最终版补充实现。概念参考 T01、T02、T03。

Q01｜如何用架构说明你的 Agent 项目解决了什么问题?

来源：**公开面经题**，美团 Agent 一面 S06、阿里 AI 全栈一面 S03。

参考回答：

我做的是基于 kbot 的企业 Agent 平台课程项目，主要解决业务人员需要在多个系统之间查询信息、核对规则、再发起操作的问题。我用了海外电商和保险两个场景验证：电商处理缺货订单、物流方案和结算差异，保险处理条款核验、赔款计算和风险分流。两套业务服务使用模拟数据，可以在本地复现完整链路。

架构上，我把它分成控制面、Runtime 和业务能力三个部分。控制面管理 Workspace，以及 Agent、Prompt、模型、Tool、Skill、KB、Eval 的配置与版本；Runtime 固定当前会话的依赖，装配上下文，通过 Eino ADK 执行模型和工具循环；业务服务负责订单、库存、案件状态及金额等确定性规则。查询过程通过 SSE 返回，敏感写操作进入人工审批，批准后由 Worker 从 checkpoint 续跑。

以订单缺货为例，Agent 查询订单和各仓库存，再结合物流与 SOP 提出方案；需要调拨时，平台暂停并创建审批，业务接口在执行时继续核验库存。这样可以把动态信息整理和受控业务执行连起来。我会用这条请求链说明自己负责的模块，并展示工具结果、审批记录和 Trace；真实 OMS/WMS 接入及业务幂等持久化仍是后续工作。

Q02｜最小 Agent Loop 的输入、迭代与结束条件是什么?

来源：**公开面经题**，字节 AI 应用一面 S02。

参考回答：

我理解最小 Agent Loop 有四个核心环节：准备输入、模型决策、执行工具、把观察结果送回模型。输入除了用户问题，还包括系统指令、当前允许的工具描述，以及与任务有关的历史和证据；这些内容决定模型能看见什么、可以申请哪些能力。

每轮模型调用后，我先判断输出是最终回答还是 tool calls。存在工具调用时，执行层按工具名找到固定版本的执行器，校验参数、权限和业务条件，再执行。结果必须通过 call ID 对应回原调用，模型才能根据成功、失败或缺失信息决定下一步。我的项目用 Eino ADK 承担循环，平台在调用前后加入权限、审批、审计和计量。

结束也有多种情况：正常给出最终答案、达到最大迭代限制、发生无法继续的错误、context 被取消，以及需要审批的可恢复暂停。我会给这些情况保留不同的状态。例如订单诊断可以查询订单、库存和物流后结束；调拨工具触发审批时，当前运行结束于等待审批，后续批准再恢复。HTTP 返回成功、流关闭和业务任务完成，需要分别判断。

Q03｜ReAct、Plan-and-Execute、Reflection 怎样选择?

来源：**公开面经题**，快手秋招 S07。

参考回答：

我会根据任务的不确定性、依赖关系和验证成本选择范式。ReAct 是模型根据当前观察选择行动，拿到工具结果后再决定下一步，适合一开始无法确定完整查询路径的任务。订单诊断就是这种情况：查完订单后，才知道要补查哪个 SKU、哪个仓库和什么物流方案。

Plan-and-Execute 会先把目标拆成计划，再按步骤执行，必要时重新规划。它适合任务较长、阶段和依赖相对清楚的情况，但计划需要结合实际工具结果更新。Reflection 则利用测试结果、规则反馈或独立评审反思上一轮产物；我会限制轮数，并要求反馈可验证，避免模型反复自我评价却没有质量提升。

我的项目主链路采用 ReAct。保险责任计算和资金操作的前置条件由确定性服务控制，模型在允许范围内检索和解释。当前没有交付通用规划器或 Reflection 优化器；如果扩展，我会用同一组复杂任务比较成功率、模型调用次数、延迟和失败类型，再判断增加规划或反思环节是否有收益。ReAct 论文是我理解交替行动范式的基础。

Q04｜Workflow 与 Agent 如何分工，如何确定自主决策边界?

来源：**公开面经题**，阿里 AI 全栈一面 S03。

参考回答：

我会先把业务步骤按确定性和风险拆开。步骤固定、前后依赖明确、需要强制执行的部分，适合放进 Workflow 或服务端状态机；需要根据自然语言和中间结果选择信息来源的部分，可以交给 Agent。这个边界最终要落实到接口和状态约束中。

在电商场景，Agent 可以决定先查库存还是物流，也可以补充检索履约 SOP；调拨数量、可用库存、订单状态则由业务接口检查。在保险场景，Agent 汇总案件事实和条款依据，规则接口计算责任与可赔上限；赔付、冻结和立案还需要审批，并满足各自前置条件。即使模型建议越过某一步，执行层也应该拒绝不满足条件的操作。

我的选择依据是失败代价和可验证性。信息整理可以允许模型探索，金额计算和状态迁移需要明确契约。当前 kbot 的 Pipeline 提供顺序协作，业务服务承担硬性门禁；更完整的跨步骤 Workflow 可以继续扩展。我会分别测试正常路径、缺少前置状态和直接调用下游工具的情况，确认自主决策始终在授权范围内。

Eino 的 Agent 与 Graph 路线辨析也讨论了预设流程与自主决策的组合。

Q05｜采用开源 ADK 后，你自己实现了哪些价值?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我选用 Eino ADK，主要考虑模型与工具循环、事件处理和中断恢复已经有成熟抽象，可以把精力放在平台和业务集成上。我理解 ChatModelAgent、Runner、ToolsNode 各自的职责，也会沿着一次工具调用确认框架与平台的边界。ADK 文档可以帮助核对接口，实际行为以项目固定依赖为准。

项目中的工程工作包括配置不可变版本、会话快照、Workspace 隔离、不同来源的工具适配、Skill 授权、敏感操作审批、审计、评测和模型额度治理。例如会话开始时固定 ToolVersion，运行时再检查权限；审批时保存框架 checkpoint，并绑定具体工具调用对应的中断点，批准后才能恢复该操作。

我会按自己的实际贡献展开其中一个模块，而不会把框架提供的能力都归到个人实现。以审批集成为例，需要验证断点持久化、正确目标恢复、重复队列消息和业务幂等之间的配合。能展示这些契约与失败测试，比只说明接入了一个 ADK 更能证明我对系统的理解。

Q06｜如何解释 Harness，并用本项目说明它的作用?

来源：**公开面经题**，4399 Agent 面经 S09、vivo Agent 面经 S10。

参考回答：

我把 Harness 理解为围绕模型循环搭建的运行支撑系统，负责让模型提出的行动变得可执行、可约束、可追踪。这个术语在不同团队里覆盖范围有差异，我在项目中主要用它指上下文装配、工具执行、权限、运行状态、恢复、观测、验证和资源限制这一组能力。

在 kbot 中，ADK 负责模型与工具循环，平台补充固定版本和会话历史，Tool Registry 把业务接口转成统一能力，Guard 和服务端授权控制边界，审批机制暂停高风险动作，Trace 和审计记录过程，Eval 检查改动是否退化。它们共同决定 Agent 能否稳定完成业务任务。

比如模型提出退款时，运行支撑层需要确认工具版本、调用参数、授权和审批状态；业务服务还要核验订单与金额。请求超时后，也要识别退款是否已发生，避免直接重试导致重复副作用。我的项目能复现审批暂停与续跑，但任意崩溃点的通用任务恢复还需要更多持久化状态，这也是我评估 Harness 完整性时会检查的部分。

Q07｜多 Agent 的路由应该根据什么决定?

来源：**公开面经题**，美团 Agent 一面 S06。

参考回答：

我会先定义角色的能力和权限，再确定路由方式。能力描述要具体到能解决什么问题、需要什么输入、会返回什么结果；权限描述要落实到工具、知识库和可访问的数据范围。只按角色名称拆分，通常不足以让路由稳定工作。

对于明确的分类，比如订单履约和保险理赔，我会优先用规则或显式入口选场景；对于混合自然语言任务，可以让主管模型根据成员描述选择专家。输入不完整或分类不明确时，我会让系统澄清，或采用受控的人工处理分支。项目有商品运营、供应链协同、理赔审核和反欺诈四类预置 Agent，Supervisor 可以将固定版本成员作为 AgentTool 调用。

路由之后仍要独立校验成员权限。例如主管把任务交给供应链 Agent，也不能因此获得保险知识库或赔付接口的访问权。我会通过单一意图、跨场景意图和缺失信息样本评估路由质量，同时观察误路由后的行为。当前没有独立的数值置信度路由模块，这类门限和兜底策略需要单独实现与标定。

Q08｜什么情况下需要 Multi-Agent?

来源：**公开面经题**，百度 Agent 实习 S01、美团 Agent 一面 S06。

参考回答：

我通常先用单 Agent 建立基线，再判断是否需要拆分。任务具有稳定专业分工、不同知识范围、不同权限或上下文过长时，Multi-Agent 才有比较明确的价值。例如理赔审核负责条款和责任，反欺诈分析负责异常线索，两者可以维护各自的工具与知识资产。

协作方式取决于依赖关系。需要动态分派时可以用 Supervisor，需要固定阶段传递时可以用 Pipeline；每个成员内部仍可使用 ReAct。拆分的代价包括额外模型调用、交接时丢失事实、结果冲突和更复杂的故障定位，所以我会定义清晰的输入输出，尤其保留案件标识、证据来源和未解决的问题。

kbot 提供 Supervisor 和顺序 Pipeline，两套行业业务也可以由单 Agent 配合 Skill 完成。我会用相同任务集对比两种方案的任务成功率、调用次数、耗时、成本和权限表现。只有角色拆分带来可验证收益时，我才增加协作层；任务简单、共享事实多或延迟要求高时，单 Agent 往往更容易维护。

Q09｜多个 Agent 并行时怎样管理状态和汇总结果?

来源：**公开面经题**，美团 Agent 一面 S06。

参考回答：

如果需要并行执行，我会先检查任务之间是否独立。读取不同数据源的查询通常有并行空间；存在前后依赖，或者都会修改同一订单、库存、案件的任务，需要明确顺序和并发控制。我还会区分子任务运行状态与业务对象状态，避免一个成员完成就被解释为整个流程完成。

在扩展设计中，我会给每个子任务分配 task ID，保存输入快照、依赖、状态、截止时间和结果契约。汇总器等待必需结果，并记录成功、失败、超时和缺失项；允许部分成功的任务可以带着缺失说明返回，关键依赖失败则停止下游操作。重试也以子任务为单位，并考虑工具调用是否有副作用。

当前项目的 Pipeline 和 Supervisor 工具调用采用顺序执行，我没有把通用并行 DAG 调度算作已有能力。如果扩展保险协作，我会先并行做只读材料核验和风险查询，再串行完成规则计算及审批，并验证部分失败、超时、取消和重复结果到达时的汇总行为。

Q10｜从用户发起提问到 Agent 回答，端到端的链路情况?

来源：**公开面经题**，字节 AI 全栈一面 S04。

参考回答：

我可以按入口、上下文装配、执行和收尾四个阶段说明。前端提交 Agent ID、消息和可选 Conversation ID，后端先认证用户、验证 Workspace 和资源归属；新会话解析当前环境对应的 Agent 版本，旧会话沿用既定版本和运行配置。

接着 Runtime 加载 Prompt、模型配置、Tool、Skill、KB 和最近的会话历史，完成输入 Guard 检查后进入 Eino ADK。模型需要业务事实时调用统一工具执行器，电商或保险服务在这里通过 HTTP 接入。工具执行前还会检查 Schema、授权和敏感标记，过程通过 SSE 返回调用、结果和回答等事件。

正常完成时，后端经过输出检查后保存回答、使用量和相关记录。遇到敏感操作，则在执行前生成审批、保存 checkpoint，并把本轮标记为等待审批；批准后 Worker 重新加载会话版本并恢复目标断点。排障时我会沿 Conversation ID、Trace 和审批 ID 找到这一整条链，再用业务服务的最终状态判断操作是否真正成功。

1.4 模型基础、Prompt 与模型治理（Q11–Q18）

代码入口：模型网关、Prompt 服务、模型配置、课程 Prompt。对应第 05、12、14 课。

Q11｜Transformer 的哪些原理直接影响 Agent 应用开发?

来源：**公开面经题**，快手秋招 S07。

参考回答：

我主要关注 Transformer 原理对应用行为的影响。注意力机制让模型结合上下文中的信息，位置表示提供顺序信息；生成模型通常按自回归方式逐 Token 输出。因此输入长度、输出长度、解码过程以及缓存都会影响一次 Agent 请求的耗时和成本。Transformer 原论文是这些概念的基础。

落到 Agent 开发，我会特别看上下文和工具调用。系统指令、历史、工具 Schema 和检索片段都会占输入空间，多轮工具结果还会持续增加上下文。即使这些内容都在窗口内，噪声、冲突信息和关键事实的位置也可能影响回答。结构化输出和工具调用能力则需要通过具体模型与业务样本验证。

在订单项目中，我通过按 Agent 绑定工具、Skill 渐进加载、限制检索片段和迭代次数控制输入规模。我没有自行训练 Transformer 或实现推理引擎；我的重点是理解这些原理为什么会造成延迟、成本和稳定性问题，再通过真实请求的 Token、调用次数和失败样本验证应用侧方案。

Q12｜Token、上下文窗口、输入预算与输出预算如何关联?

来源：**课程延伸题**。

参考回答：

我会先把 Token 和字符区分清楚。Token 是分词器处理的单位，同样长度的中文、英文、订单号或 JSON，Token 数量可能不同。上下文窗口限制的是模型可处理的序列规模，输入和输出之间怎样分配，还需要看具体模型接口和限制。

做预算时，我会统计系统指令、工具描述、历史、检索结果、本轮问题和模型特有开销，再为预期输出留空间。如果超预算，我会按任务价值裁剪：先去掉无关历史和重复片段，再缩短工具结果，保留订单号、案件号、金额、时间和审批状态等关键事实。摘要和原始证据也要建立关联，便于复核。

项目当前按最近 10 条历史 user/assistant 消息构造上下文，这能限制历史条数，但一条超长 JSON 仍可能占满窗口。完整 Token 预算器是我的扩展方向。我会用长条款、超长工具结果和多轮案件测试，确认裁剪后仍能正确识别事实与状态，同时观察输入 Token、完成时间和任务成功率。

Q13｜KV Cache 有什么作用，与业务缓存怎样区分?

来源：**公开面经题**，阿里 AI 全栈一面 S03。

参考回答：

KV Cache 的作用是复用自回归解码中已经计算的 Key 和 Value。生成下一 Token 时，可以减少对已有序列的重复计算；它通常由推理服务管理，具体内存和速度取决于模型结构、上下文长度、批处理及实现。我使用模型接口时，需要了解它对延迟和容量的影响。缓存文档解释了这层原理。

应用缓存保存的则是配置、Embedding、检索候选或业务查询结果。我需要自己定义缓存键、权限范围、有效期和失效条件。例如知识检索缓存应考虑 Workspace、KB 内容与模型版本；库存和赔付状态变化快，缓存策略要更谨慎。缓存命中也不能绕过当前用户的授权校验。

我的项目没有管理推理引擎内部的 KV Cache。对于应用缓存，我会先定位实际热点，再判断是否适合缓存；配置可按不可变版本复用，业务结果必须考虑时效和副作用。我会分别报告模型侧推理优化与应用缓存的效果，避免把两种机制混成同一个性能改动。

Q14｜Prompt 调优怎样避免凭感觉修改?

来源：**公开面经题**，快手秋招 S07。

参考回答：

我的 Prompt 调优流程是先建立失败样本和基线，再做小步改动。先明确任务怎样算成功，例如工具是否选对、参数是否完整、证据是否支持结论，以及未审批时有没有声称操作已完成。然后结合 Trace 判断问题来自 Prompt、检索、工具结果还是上下文缺失。

确认属于 Prompt 问题后，我会调整角色职责、任务目标、可信事实来源、工具说明、状态约束和输出结构，并加入少量代表性示例。比如退款场景明确区分建议退款、等待审批和已执行，工具失败或状态未知时也要准确表达。项目把 System Prompt 与 User Prompt Template 分开管理，任务变量通过 Schema 校验，版本由 Agent 快照固定。

验证时我会固定模型、工具和数据集，尽量一次只改一个因素；随机性明显时重复运行，并保留未参与调优的测试集。比较任务成功率、关键违规、Token 和延迟后，再决定是否发布。我目前不会报一个没有实验记录支撑的提升比例，能展示的应包括失败样本、改动版本和逐例结果。

Q15｜RAG、SFT、LoRA 各自解决什么问题?

来源：**课程延伸题**；相关训练考点见 S05、S07。

参考回答：

我会先看问题出在知识还是行为。RAG 在推理时检索外部资料，适合需要更新、引用和访问控制的知识；SFT 利用监督样本训练模型的行为；LoRA 是参数高效适配方法，可以用于 SFT 等训练过程。这几个概念可以组合，选择时需要看数据、成本和验证结果。RAG与 LoRA论文分别说明了相应方法。

我的项目中，履约规范和保险条款适合放进 KB，订单、库存、保单和风险状态通过工具获取。即使经过训练，模型也无法仅凭参数知道当前库存或某次审批的结果，所以实时事实和写操作仍需要业务接口。当前课程交付重点是 RAG、工具和执行治理，没有交付模型训练流程。

如果工具选择或输出习惯在多种 Prompt 方案下仍不稳定，我会先确认有足够高质量标注数据，再设计训练对照实验。评估除了文本表现，还要看工具参数、权限和业务最终状态；知识更新与可追溯问题仍由检索和版本治理处理。

Q16｜为业务 Agent 选模型应该看哪些指标?

来源：**课程延伸题**。

参考回答：

我选模型会从业务任务出发，先设质量门槛，再比较成本和延迟。Agent 特别需要稳定的工具选择、Schema 合法率、参数准确性和多轮任务完成能力；同时还要看上下文容量、中文及跨语言表现、输出格式，以及数据能够送到哪些部署环境。

项目用 Provider、Deployment 和 Profile 管理模型接入与调用策略。我会拿相同的订单诊断、结算核对、正常理赔和高风险处置任务做对照，固定 Prompt、工具与输入。除了最终回答，还检查调用轨迹、审批行为、失败恢复和业务结果，再统计首响应、任务总耗时、Token 和单任务成本。

课堂链路可以接真实模型，完整演示也有 Mock 路径。Mock 适合验证接口和流程，真实模型选型必须重新测。对于海外电商我会补英文与混合语言订单，对于保险补长条款和缺失材料样本。达到质量和数据约束的候选，才进入价格与吞吐比较。

Q17｜模型限流或不可用时怎样重试和降级?

来源：**课程延伸题**。

参考回答：

我会先分类错误，再决定重试还是停止。网络抖动、限流和部分服务异常可能是临时问题，可以做有上限的重试；鉴权失败、无效参数或不支持的能力需要修正配置。重试还要受整体截止时间约束，避免用户已经取消，后台仍在不断请求。

模型降级时，我会检查备用模型是否满足相同的数据路由、工具调用、上下文和质量要求。kbot 通过 Model Profile 将候选装配到 ADK 的重试与故障转移配置中，并结合额度治理。降级后仍需要回归真实任务，尤其是复杂 Schema 和多轮工具调用，不能只看能否返回文本。

我会把模型生成重试和业务工具重试分开处理。订单只读查询通常可以按策略重试；退款请求发出后超时，结果可能已经在远端生效，需要查状态或使用同一幂等键。项目已有模型重试和备用配置，具体第三方工具的重试、查单与补偿策略还要按业务契约补齐。

Q18｜Prompt、Tool 与模型配置发新版后，旧会话如何稳定运行?

来源：**课程延伸题**。

参考回答：

我用不可变版本和会话快照保持运行配置稳定。创建新会话时，平台解析环境当前指向的 Agent 版本，并把 Prompt、Tool、Skill、Model Profile 等依赖固定下来；旧会话继续使用自己的依赖，不会因为管理员发布新配置就突然更换工具 Schema。

比如保险审核进行到一半时，approve_claim 的参数发生变化，新会话可以使用新版本，原会话和审批续跑仍需按旧版本装配。项目支持按 ToolVersion 构建执行器，并在恢复时加载会话对应的 AgentVersion。旧版本所需适配器和凭据兼容性也要保留，否则仅保存 ID 无法保证继续运行。

我还会说明复现范围：固定配置能减少行为漂移，KB 内容、实时业务状态和动态安全策略仍可能变化，模型本身也有随机性。若要更完整复核一次理赔，还应记录检索片段、条款及规则版本、工具输入输出和审批记录。配置版本稳定与业务事实更新，需要同时处理。

1.5 上下文与记忆（Q19–Q24）

代码入口：历史装配 buildMessages、会话领域对象。对应第 06、14 课。

Q19｜Agent 消息包含哪些角色，Tool call 怎样对应结果?

来源：**公开面经题**，字节 AI 应用一面 S02。

参考回答：

我在应用中主要接触 system、user、assistant 和 tool 消息，具体角色以模型协议为准。system 提供运行指令，user 提出任务，assistant 产生回答或工具调用，tool 返回执行结果。工具调用通常带 call ID，返回消息必须对应这个 ID，模型才能知道结果属于哪次请求。

一次请求里可能包含多个调用，所以我会关注配对和顺序：工具名相同也不能靠名称猜对应关系；结果缺失、重复或插错位置，可能让模型误解执行状态。项目使用 Eino schema 表达消息，ADK 管理当前运行中的工具消息和循环。

跨轮对话时，kbot 的 buildMessages 恢复最近的 user/assistant 历史；审批续跑则使用独立的框架 checkpoint。普通对话历史包含用户可见内容，无法完整表达执行位置、工具调用和中断状态。因此恢复一次敏感操作时，我会查 checkpoint、审批及业务结果，不会仅凭聊天文本判断它还需执行什么。

Q20｜短期记忆与长期记忆在项目中分别如何实现?

来源：**公开面经题**，字节 AI 应用一面 S02、阿里 AI 全栈一面 S03。

参考回答：

我把短期记忆理解为服务当前任务的上下文，包括近期消息、当前约束和运行状态；长期记忆用于跨任务复用经过确认的偏好、事实或经验。两者的保存周期和可信度不同，长期记忆尤其需要来源、权限、更新时间、纠错和删除机制。

项目最终版持久化会话，新一轮会取最近 10 条历史 user/assistant 消息。这里是 10 条消息，不能直接等同于 10 轮对话；工具中断恢复还有独立 checkpoint。当前没有实现完整的长期语义记忆产品，我会准确说明这一点。

如果扩展电商商户偏好，我会保存经过确认的运输偏好和适用范围，检索时校验身份与时效。实时库存、保单效力、审批结果则从权威业务接口读取，避免历史记忆覆盖新状态。我还会设计错误记忆更正和撤销流程，并测试跨商户访问、过期偏好和与当前业务结果冲突的情况。

Q21｜长对话怎样压缩，如何避免丢失关键事实?

来源：**公开面经题**，美团 Agent 一面 S06。

参考回答：

我会先区分必须精确保留的任务状态和可以压缩的讨论内容。订单号、案件号、金额、时间约束、已经执行的操作、审批 ID 和证据来源，应作为结构化事实保留；重复讨论、无关解释可以摘要。只保留一段自由文本总结，容易丢掉条件、否定词或状态差异。

方案上可以组合近期窗口、结构化任务状态和历史摘要。摘要需要记录覆盖范围、来源与生成时间，并在关键写操作前重新查询业务事实。比如摘要中的"计划退款"必须保留待审批属性，不能在压缩后变成"已退款"；历史库存也不能替代执行时的库存查询。

kbot 当前使用最近 10 条消息窗口，自动摘要属于扩展。我会用长对话对照测试，检查案件身份、金额、否定条件、时区和审批状态是否被保留，同时衡量压缩比例与任务成功率。若摘要无法支撑关键判断，系统应补查原始证据或请求澄清。

Q22｜上下文窗口变大后，还需要上下文工程吗?

来源：**课程延伸题**。

参考回答：

上下文窗口扩大后，我仍然需要上下文工程。窗口容量解决的是能放多少内容，实际任务还受到相关性、信息冲突、成本和延迟影响。把所有历史、工具描述和文档放进去，可能稀释当前案件的关键信息，也会增加每轮调用开销。

我的做法是先确定当前任务需要哪些资产，再控制输入的内容和顺序。项目通过 Agent 工具绑定、Skill 渐进加载、KB Top-K 和历史窗口限制输入；保险审核时优先保留本案保单、事故时间、适用条款和工具结果，避免其他案件或过期规则混入判断。

进一步扩展时，我会增加 Token 预算和工具结果处理，根据任务优先级选择片段、摘要或补查。验证也要覆盖长上下文中的干扰、相互矛盾的资料和靠后出现的关键事实。我的目标是让上下文足够完成任务且证据清楚，并用成功率、Token 和总耗时评价方案。

Q23｜工具返回十万行日志或超长 JSON，该怎样处理?

来源：**公开面经题**，4399 Agent 面经 S09。

参考回答：

我会在工具和上下文两层处理大结果。工具侧先支持条件过滤、分页和聚合，限制响应体；模型只拿到与当前问题相关的摘要、统计和结果位置。完整日志或 JSON 可以保存为有权限和有效期的产物，再提供受控的分段读取能力。

返回结构里我会明确总量、筛选条件、当前范围和截断标志。这样模型知道自己看到的是部分数据，无法直接据此宣称"所有记录均正常"。以结算为例，先按账期、商户和差异类型聚合，再读取异常订单明细，比把十万行账单一次送进模型更容易核验。

项目已有工具响应和沙箱输出大小限制，自动日志索引及产物按需检索仍需扩展。我会测试超大响应、单条异常位于截断之后、分页失败和未授权读取等分支；同时确认截断后回答能说明证据范围，避免大小限制掩盖关键失败。

Q24｜长期记忆和当前工具结果冲突，信任哪一个?

来源：**课程延伸题**。

参考回答：

我会按来源权威性、适用对象和时间判断冲突。当前库存、订单状态和赔付状态优先向业务系统核验；长期记忆适合提供背景或偏好。两条信息冲突时，我会保留冲突说明，必要时澄清对象，而不会让模型自行把不同时间的事实拼成结论。

比如历史摘要说洛杉矶仓有货，本轮库存接口返回可用量为零，我会先确认 SKU、仓库、时间和口径是否一致，再按当前结果重新规划。保险场景同样要确认产品版本和事故时间，避免拿其他案件的记忆判断本案责任。

我的项目当前业务事实来自工具，长期记忆系统还属于扩展。如果实现，我会给记忆附来源、更新时间、适用范围和撤销状态，只有经过确认的信息才能写入；冲突时可标记过期或等待复核。验证会覆盖同对象状态变化、同名不同对象和过期记忆，确认写操作最终依据的是受控业务事实。

1.6 Tool、Skill、MCP 与 A2A（Q25–Q36）

代码入口：统一执行器、Skill Runner、MCP 集成、A2A 集成、Endpoint Policy。对应第 03、07–09、13、21 课。

Q25｜Function Calling 从模型输出到业务执行有哪些步骤?

来源：**课程延伸题**。

参考回答：

我会把 Function Calling 讲成一个由模型申请、服务端执行的过程。首先平台向模型提供工具名、用途和参数 Schema，模型根据任务输出工具名、参数及 call ID。此时还没有业务副作用，只是产生了调用意图。

接下来执行层找到对应工具版本，检查它是否在 Agent 和当前 Skill 的授权范围内，再落实 Schema、KB、网络及业务校验。敏感工具在业务执行前进入审批，批准后才有机会调用实际接口。执行结果通过 call ID 放回上下文，让模型继续判断下一步或形成最终回答；错误也要准确返回，不能被当成成功事实。

在我的电商项目里，调拨工具要求显式传 SKU、源仓、目标仓、数量和幂等键。业务接口核验库存及数量，身份与资源授权由受控后端决定。我会特别防止模型自行填一个租户字段就获得权限，也会区分参数合法、审批通过和业务执行成功这三个判断层次。

Q26｜如何设计统一 Tool Executor?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我设计统一 Tool Executor 时，会把模型可见的描述和实际执行能力分开。模型看到名称、用途和参数 Schema，执行层接收 context 与 JSON 参数，返回结果或错误。这样 REST、MCP、内部 SDK、代码执行和 A2A 都能通过适配器进入同一条 Runtime 链路。

项目的 Registry 按固定 ToolVersion 构造 BuiltTool，其中包含工具描述、执行器、敏感标记、网络需求、KB 范围和审批展示信息。Schema 校验包在执行器外层，Runtime 统一做授权、Guard、审批和审计；不同协议适配器处理自己的连接、超时与响应约束。业务服务只需要维护清晰的接口和业务规则。

我不会给所有工具套同一种重试策略。只读查询、可幂等写入和结果未知的资金操作，需要不同处理。以 approve_claim 为例，统一层能管理权限与审批，业务端仍需核验案件、金额和幂等结果。我会用非法参数、未授权、超时和重复执行测试，确认不同工具来源都遵守同一组治理边界。

Q27｜工具层兜底要覆盖哪些失败?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我会把工具失败分成参数错误、权限拒绝、业务拒绝、临时技术故障和结果未知五类。识别依据优先用错误码、结构化结果、HTTP 状态及调用阶段，字符串只作为辅助；并确认请求是否已经发到远端，因为这直接影响能否重试。

参数错误可以把允许修正的信息交回模型，例如缺少订单号；权限拒绝应停止相应动作，不能引导模型换一种方式绕过。库存不足属于业务拒绝，可以重新查询和生成方案。临时网络故障可以在截止时间内有界重试；退款超时但远端可能已受理，则先查单或复用幂等键，必要时转人工核对。

项目已有 Schema、授权、超时和业务状态门禁，具体错误分类与重试规则需要逐个工具确认。我还会让回答区分失败、待确认和完成，保留调用标识及原因。验证时尤其覆盖"业务已写入、响应丢失"这种分支，因为仅根据本地超时重新发起，可能产生重复副作用。

Q28｜工具太多导致选择困难和 Prompt 过长怎么办?

来源：**课程延伸题**。

参考回答：

我会先按职责减少工具集合，再考虑动态选择。工具过多会增加上下文长度，也会让相近工具的用途更难区分。第一步是让 Agent 只绑定完成其职责所需的工具，第二步用清晰名称、描述和参数说明降低歧义，第三步用 Skill 聚焦当前任务。

项目里商品运营、供应链、理赔和反欺诈分别绑定场景能力，Skill 再通过 allowed-tools 限制激活后的范围。比如结算任务只需要查账和申诉相关工具，保险赔付接口无需进入这个 Agent 的可见集合。工具描述还应写清适用条件、查询与写入差异，以及常见失败含义。

如果未来有上百个工具，我会评估工具检索加二阶段选择，但要测试关键工具漏召回、近义描述混淆和延迟。动态检索属于当前项目的扩展方向。无论工具是否被模型看到，服务端的版本绑定和授权检查仍需生效，我会分别验证选择质量与越权拒绝。

Q29｜模型输出合法 JSON，是否足以保证业务安全?

来源：**课程延伸题**。

参考回答：

合法 JSON 只能说明语法可解析。我的检查会继续覆盖 Schema、资源授权和业务规则。Schema 可以约束类型、必填项、枚举、范围及额外字段；跨字段关系、实时库存、保单状态和金额上限，需要业务服务结合当前事实判断。

比如 `quantity=-1` 可能是合法 JSON，但违反调拨数量要求；一个结构完全正确的案件 ID，也可能属于其他 Workspace。即使金额是正数，还要核验币种、可赔上限和案件阶段。敏感动作通过审批后，执行时仍要做这些检查，防止等待期间事实已经变化。

项目在统一执行器层做 Schema 校验，Runtime 限制 Tool、KB 和网络，业务模拟器做状态与金额门禁。接入真实系统时，我会补齐资源归属、事务及持久化幂等。测试也会分层：语法正确但类型错误、Schema 通过但跨字段错误、参数合法但无权限，以及批准后状态冲突，分别验证拒绝位置和结果表达。

Q30｜Skill 为什么渐进式加载，怎样落实权限?

来源：**公开面经题**，阿里 AI 全栈一面 S03。

参考回答：

我采用 Skill 渐进加载，主要是为了让模型先判断任务是否相关，再读取详细流程。初始只提供名称和触发描述，选中后通过 skill 工具加载正文，减少无关操作手册占用上下文。Skill 描述完成一类任务的步骤、证据和异常处理，Tool 提供可执行的单项能力。

kbot 使用 Eino Skill Middleware，把已发布、已绑定的固定版本适配成 Backend。加载时检查网络需求并记录触发来源；激活后收窄可见工具，真正调用时再核验 allowed-tools、allowed-kbs 和 Agent 授权。显式选择 Skill 只改变触发方式，不会自动扩大权限。

以结算 Skill 为例，正文指导先查账、核对差异、准备证据，再申请申诉；申诉工具仍需审批，金额由业务结果确认。我会测试自动命中、显式触发、缺少绑定、禁网和越权 KB 等分支。渐进加载能减少上下文，流程遵循程度仍需评测，强制业务前置条件继续放在执行层。

Q31｜怎样保证 Agent 按指定流程办事?

来源：**公开面经题**，京东后端实习 S08。

参考回答：

我会先把流程拆成建议步骤和必须满足的业务条件。建议步骤可以通过 Prompt、Skill 和示例指导模型；需要强制满足的依赖，要落实在状态机、接口校验或确定性 Workflow 中。这样即使模型漏掉描述里的步骤，执行层仍能阻止越序操作。

在保险项目里，理赔查询、条款检索和解释可以由 Agent 组织；赔付、冻结和调查立案要满足各自状态与风险条件，并进入审批。审批确认具体操作和参数，业务接口在执行时还会检查当前状态。需要修改金额或对象时，应重新申请对应动作的批准。

当前项目已有调查、赔付的业务门禁和敏感工具审批，通用跨步骤 Workflow 还可以扩展。我验证流程时会故意直接调用下游工具、跳过查询、使用过期状态和重复提交。只看正常对话是否按步骤输出还不够，关键是违规执行尝试能否被服务端拒绝，并留下可解释的原因。

Q32｜MCP 的 stdio、Streamable HTTP 与 SSE 分别是什么关系?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我会先区分传输协议和承载格式。按课程固定的 MCP 2025-11-25 规范，标准传输包括 stdio 和 Streamable HTTP。stdio 由客户端启动子进程，通过标准输入输出交换 JSON-RPC 消息；日志应走 stderr，避免污染协议输出。

Streamable HTTP 通过 HTTP 请求交换消息，响应可以是 JSON，也可以使用 SSE 形成消息流。SSE 在这里承担服务端事件交付，和 Agent 前端显示逐字回答的业务语义需要分别理解。旧版 HTTP+SSE 传输与当前 Streamable HTTP 的端点和会话规则有差异，接入时要明确版本。传输规范是我核对这些行为的依据。

项目远程工具使用 Streamable HTTP，stdio 适配和测试有保留，但 App/Worker 控制面拒绝直接配置本机进程启动。若需要生产使用 stdio，我会放到独立受控 Runner，管理可执行文件、参数、凭据和生命周期，并验证超时、退出及清理，避免把任意进程能力开放给普通配置。

Q33｜MCP 工具发现、调用与鉴权有哪些工程细节?

来源：**课程延伸题**。

参考回答：

我会按初始化、发现、调用和关闭四个阶段接入 MCP。初始化阶段协商协议版本与能力，并完成初始化通知；发现阶段获取 tools 列表，匹配平台允许暴露的工具；调用阶段传参数并处理协议及工具级错误；最后管理连接和会话生命周期。生命周期规范提供了这条顺序的依据。

工程上，我还会检查认证、服务身份、会话标识、超时和取消。凭据由受控配置注入，网络连接经过地址策略，工具描述与结果都按外部内容处理。对端宣称某项能力，仍需要契约测试确认；协议能连通，也不直接代表用户有权操作对应业务资源。

项目发现工具后筛选固定配置的目标工具，并统一进入 Runtime 授权和审计链路。当前没有注册 sampling、elicitation 等反向处理能力，我会明确限制。测试包括初始化失败、工具不存在、Schema 不匹配、会话失效、超时和恶意输出，确认每种问题都能被准确暴露。

Q34｜A2A 与 MCP 的接入职责怎样划分?

来源：**课程延伸题**。

参考回答：

我会按接入对象划分职责。MCP 为工具等能力提供标准接口，A2A 为 Agent 之间的能力发现、消息和任务交互提供协议。远端 Agent 可以在内部使用自己的模型、工作流甚至 MCP 工具，本地系统主要依据它的对外契约协作。

在 kbot 中，远端 A2A 能力包装成一种 Tool。执行器读取 AgentCard，选择 JSON-RPC 接口，把输入归一化成 SendMessage 请求，再把响应交回本地模型循环。第 21 课用外部物流 Mock 演示这种接法；本地退款或调拨权限仍由本地工具、审批和业务服务控制。

课程适配范围固定在 A2A v1.0.1 的相应调用上，完整任务查询、流式订阅、推送和 Card 签名验证需要继续实现。我会先确定对端版本、鉴权、任务状态及结果格式，再做契约测试。对于长任务，收到响应后还需判断是否真正完成；仅能发出 SendMessage，无法代表完整异步生命周期已经接通。A2A 规范可用于校核协议对象。

Q35｜Tool 为什么需要独立版本与发布门禁?

来源：**课程延伸题**。

参考回答：

我给 Tool 独立版本，是因为它的 Schema、地址、鉴权和业务语义都会影响会话执行。只保存工具名称或“当前版本”，发布后旧会话可能突然调用另一份参数契约，审批续跑也可能与原来批准的操作不一致。

项目用不可变 ToolVersion 管理配置，Agent 快照引用具体版本，Runtime 通过 BuildByVersion 构造执行器。发布前还需要同版本试调，确认 Schema 和服务行为匹配。对于写工具，我会用 dry_run 验证输入及部分业务条件，再在受控环境验证真正的审批和副作用链路。

比如 approve_claim 修改金额字段时，我会新建版本，检查依赖的 Skill、Agent 和评测样本，并给旧会话保留兼容路径。发布门禁可以降低配置错误，业务系统状态变化和凭据失效仍需运行时处理。我也会测试旧会话和等待审批的任务在新版本发布后是否继续使用正确契约。

Q36｜HTTP Tool 如何防止 SSRF，代码工具如何隔离?

来源：**课程延伸题**。

参考回答：

我会分别处理 HTTP 出网风险和代码执行风险。HTTP 工具侧需要校验 scheme、userinfo、主机、解析后的 IP 和实际连接目标，限制特殊与未授权私网地址，并对重定向逐跳检查。只检查配置字符串，无法覆盖 DNS 变化或跳转后的地址。

项目的 Endpoint Policy 和运行时连接策略提供这些基础约束；课堂需要访问的内网服务使用显式 allowlist。代码工具则进入独立 Sandbox Runner，每次用一次性容器执行，限制网络、根文件系统、用户权限、CPU、内存、PID、时长和输出，并清理取消或超时的容器。

我会测试私网绕过、重定向、域名解析变化，以及代码访问网络、写根目录和资源耗尽等分支。Docker 容器与宿主共享内核，所以实际部署还要管理镜像、Runner 控制权限和宿主环境；高风险场景可以评估更强隔离。真实业务凭据也应通过受控接口使用，避免进入代码执行容器。

1.7 RAG 全链路与知识治理（Q37–Q48）

代码入口：ingest、切分、内存检索、持久化检索、Embedding。对应第 10、11、13 课。技术依据：PostgreSQL 全文检索、pgvector、RAG 论文。

Q37｜从文档导入到答案生成，RAG 的完整链路是什么?

来源：**公开面经题**，字节 AI 全栈一面 S04、美团 Agent 一面 S06。

参考回答：

我会按入库和查询两条链路说明。入库时，Connector 获取文档，ingest 依次解析、切分、生成 Embedding、写入索引，并记录文档来源、内容 Hash 和处理状态。项目最终版由 Worker 执行入库，片段和向量保存在 PostgreSQL，API 与 Worker 共享这份索引。

查询时，先校验用户、Workspace 和允许访问的 KB，再把问题向量化，同时做关键词与向量召回；候选通过 RRF 融合后，选取 Top-K 片段交给模型。返回片段保留文档和 chunk 标识，回答时要根据证据解释，并在证据不足时澄清。检索相关性还需结合条款版本、时间和适用范围判断。

我的电商 KB 存履约和结算 SOP，保险 KB 存条款和审核规范。实时库存、订单及案件状态通过业务工具读取。排查错误时，我会依次看原文是否存在、切分是否破坏条件、候选是否召回、排序是否靠前以及模型是否正确使用证据，分别定位入库、检索和生成问题。

Q38｜Chunk 大小与 overlap 怎样确定?

来源：**公开面经题**，百度 Agent 实习 S01、美团 Agent 一面 S06。

参考回答：

我确定 Chunk 参数时，会先以业务语义完整性作为基准。片段太小容易拆开条件和结论，太大则会引入噪声、增加上下文；overlap 可以缓解边界丢失，但也会增加重复候选和存储。参数需要通过业务查询评测，无法仅凭一个通用默认值决定。

保险条款中，我希望适用条件、例外和定义能一起被召回；电商 SOP 则要保留操作步骤与前置状态。项目的 chunkText 优先按段落拼接，超长段落硬切并保留尾部重叠，默认 size 为 500。实现里混用了字节长度判断和 rune 切分，因此我不会把它直接描述成 500 Token，也会专门测试中文和中英文混排。

实验时，我会固定文档与查询集，比较不同长度、重叠和 Top-K 下的证据 Recall、答案正确率、Token 与延迟。还会检查跨段引用、长列表和重复片段。如果复杂条款经常被切坏，我会扩展按标题、条款编号和结构切分，再用同一评测集确认收益。

Q39｜为什么要混合检索，项目具体用了什么实现?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我使用混合检索，是因为精确表达和语义改写需要不同信号。SKU、条款编号和专有名称适合关键词匹配；用户用另一种说法描述缺货、理赔责任时，向量检索可以补充语义候选。两个通道各有漏召回情况，我会融合后再评估效果。

项目有两条实现路径：教学和内存检索器包含 BM25 与向量召回；最终 PostgreSQL 路径使用 ts_rank_cd 全文排序、pgvector 余弦距离及 RRF。SQL 里虽然有名为 bm25 的 CTE，实际算法需要按代码说明。当前全文检索使用 simple 配置，中文分词和跨语言质量还需专项验证。

比如用户输入具体条款号，我希望关键词通道保住准确候选；用户问“事故发生在生效前还能赔吗”，则需要检索相应责任条件。我会分别统计精确词、语义改写和干扰样本的召回，再比较单通道与融合结果。订单实时状态仍从业务接口读取，混合检索负责资料证据。

Q40｜RRF 怎样合并不同检索结果?

来源：**课程延伸题**。

参考回答：

RRF 是按排名融合候选的方法。我使用的形式是 `RRF(d)=Σ 1/(k+rank_i(d))`，其中 rank 表示片段在某个通道中的名次，k 是平滑常数。片段没有出现在某个通道里，该通道就不贡献分数；同一片段在多个通道都靠前时，综合得分通常更高。

它的好处是无需把全文分数和向量距离直接放在同一量纲上比较。比如一个片段在关键词通道排第 1、向量通道排第 3，k=60 时贡献就是 `1/61+1/63`。项目 SQL 各取最多 50 个候选，按 chunk ID 合并排名贡献，再取最终 Top-K。

我会把候选召回和融合排序分开验证：如果正确片段没有进入任一候选集合，RRF 无法补救；如果大量近重复片段进入，也可能挤占上下文。k=60 是项目默认值，我会用精确标识、语义问法和噪声样本比较参数，观察证据排名与最终答案质量，避免只凭采用融合就宣称效果提升。

Q41｜Rerank 放在哪里，当前课程接了重排模型吗?

来源：**课程延伸题**。

参考回答：

Rerank 通常放在多通道召回与融合之后、选择生成上下文之前。前面的检索优先保证候选覆盖，重排再对问题与候选的相关性做更细判断。Cross-Encoder 可以联合处理问题和片段，代价是新增计算、延迟和服务依赖。重排文档提供了这种两阶段思路。

当前项目内存检索器有词重叠的轻量排序，最终 PostgreSQL 路径直接返回 RRF 排序结果，没有接独立神经重排服务。我会清楚说明已实现范围，不会把任何一次排序都称作接入了 Rerank 模型。

如果保险长条款候选包含很多相似但适用条件不同的片段，我会先保留较多候选，再尝试重排选少量证据。实验固定召回集合，比较证据排名、答案正确率和 P95，同时评估服务失败时是否回退到 RRF。若正确证据本来未被召回，我会先修检索，重排只能处理已经进入候选的内容。

Q42｜Embedding 模型怎样选，更换后为什么要重建索引?

来源：**课程延伸题**。

参考回答：

我选 Embedding 模型会关注业务术语、语言、文本长度、吞吐、成本、维度和实际召回质量。文档与查询必须使用兼容的模型、预处理和编码方式；某些模型区分 query 与 document 编码，也要按其约定使用。维度相同只满足存储形状，并不证明两个向量空间相同。

kbot 通过 Embedder 的 Embed 和 Dim 抽象接入，本地哈希实现用于离线链路，另有真实服务适配。当前数据库 migration 使用 vector(1536)，更换维度要同步改存储。即使维度不变，换了不兼容模型，也需要对文档重新向量化，避免查询和文档表示失配。

迁移时，我会建立新索引或独立版本，批量生成文档向量，用固定业务查询对比 Recall、答案和延迟，达标后切换，并保留回滚路径。这是需要落地的迁移方案。项目的 Mock 或哈希向量可以验证流程，真实语义效果仍要接真实模型测量。

Q43｜HNSW 与精确检索怎样取舍?

来源：**课程延伸题**。

参考回答：

我会用精确检索作为质量基线，再评估 HNSW 是否满足性能要求。精确检索检查符合范围的向量，数据量小时实现和验证比较直接；HNSW 通过图结构做近似搜索，在速度、召回率、内存和构建成本之间取舍。pgvector 文档说明了相关索引和参数。

我主要关注 M、ef_construction 和 ef_search：它们分别影响图连接规模、构建搜索范围和查询搜索范围，调整后要结合数据与过滤条件测试。项目 migration 已创建余弦 HNSW 索引，但查询是否实际使用它，需要看执行计划，不能只根据存在索引就推导出性能。

验收时，我会在相同 KB 和权限过滤下，以精确结果为参照统计 Recall@K，再记录 P95、内存、写入和构建成本。多租户过滤可能影响候选数量，也要检查召回不足。当前项目没有可直接引用的大规模检索压测结果，我会报告实测数据规模和配置，保留结论的适用范围。

Q44｜知识库频繁更新怎样保证一致性和可回溯?

来源：**课程延伸题**。

参考回答：

我会把更新一致性分成单文档替换和整库发布两个层次。单文档需要稳定 ID、内容 Hash、处理状态和幂等任务；新内容解析、切分和向量化成功后，再替换旧片段。这样可以避免处理中间状态直接暴露给查询。

项目持久化 Index 在事务里删除该文档的旧片段，再插入新片段，保证这一批替换的原子性。内容 Hash 和处理记录用于识别变化与排查失败。但整库原子切换、并发版本冲突、历史索引快照和条款有效时间过滤，还需要额外设计，不能从单文档事务推导出全部版本治理能力。

在保险场景，我会保留产品与条款版本、适用时间和案件事故时间，查询时选择适用资料；仅拿“当前最新条款”可能得出错误结论。若扩展整库版本发布，我会先构建并验证新索引，再切换查询指针，测试入库失败、重复任务和更新时并发查询，确保结果来源可回溯。

Q45｜多轮查询和多跳 RAG 怎样处理?

来源：**课程延伸题**。

参考回答：

我会先解决多轮指代，再处理多跳依赖。用户问“这个还能赔吗”时，需要从上下文确认“这个”对应哪个案件、产品和时间；无法确认就澄清。查询改写应只补齐已有事实，不能替用户编造事故原因或责任条件。

多跳任务可以按证据依赖拆开。例如先查案件和保单产品版本，再检索该版本责任条款，最后调用责任计算接口，并核对结果是否得到证据支持。每一步都记录来源、缺失项和下一步所需信息，同时限制查询次数，防止反复检索没有进展。

项目可以由 Agent 组合业务工具与 search_knowledge_base 完成多步任务，当前没有独立通用的 Query Rewrite 或多跳规划模块。如果扩展，我会用跨轮指代、多个同名对象、版本变化和中间证据缺失样本验证；评价不仅看最后的文本，还看每一步是否引用了正确对象和真实证据。

> 多轮指代需要进行指代消解与补全，将带有“他/她/它/这个/那个/当前”等指代不明的词汇，根据上下文补全为包含完整实体名、时间、背景的**独立查询**。

Q46｜如何控制 RAG 和 Agent 回答中的幻觉?

来源：**公开面经题**，美团 Keeta 一面 S05。

参考回答：

我会沿检索、生成、执行和评测四层控制幻觉。先检查资料是否正确、适用版本是否匹配、关键证据能否被召回；再让模型区分事实、推断和缺失信息，引用能支持结论的片段。检索到相关文本，也需要确认其中条件是否适用于当前对象。

项目中，订单金额和库存来自工具，保险可赔上限由 evaluate_coverage 返回，模型主要负责组织和解释。敏感动作由审批与业务门禁控制，避免一句错误回答直接变成资金操作。证据不足时，我会让系统补查、澄清或交人工，回答中准确标明待确认状态。

验证时我会加入错误前提、过期条款、工具失败、引用不支持结论和缺失材料样本，分别看事实正确性与执行行为。Guard、Prompt 和引用可以降低风险，效果仍要靠测试与实际记录证明。我也会核对业务最终状态，防止模型把“申请冻结”表述成“已经冻结”。

Q47｜Agentic RAG 与固定检索链路如何选择?

来源：**公开面经题**，美团 Keeta 一面 S05。

参考回答：

我会根据问题是否需要动态补充证据选择链路。固定 RAG 按预设查询和生成步骤运行，成本、延迟和失败位置更容易控制；Agentic RAG 让模型根据中间结果决定查什么、查哪个来源以及是否继续，适合跨数据源或多阶段问题。

在 kbot 中，search_knowledge_base 是受控工具，可以与订单、保单和风险查询组合。比如保险任务先获得产品版本，再检索适用条款；发现缺少事实后继续查询案件。这个过程仍受最大迭代、KB 授权、网络策略和审计约束，模型不能随意访问所有资料。

我的做法是先用固定链路建立简单问题基线，再用复杂案例验证动态检索的收益。比较时会记录证据 Recall、任务成功率、检索次数、Token 和耗时，并检查重复查询与错误扩展问题。单次检索足够回答的任务可以保留固定链路，动态能力用在确实需要补充信息的部分。

Q48｜知识库多租户隔离为什么要在召回前落实?

来源：**课程延伸题**。

参考回答：

我会在召回之前检查授权，因为片段一旦进入模型上下文、Trace 或缓存，就已经扩大了接触范围。事后在答案中隐藏内容，无法撤回上游已经看到的数据。权限判断需要覆盖用户所属 Workspace、Agent 绑定的 KB，以及激活 Skill 的 allowed-kbs。

项目把知识资产绑定到 Agent 和 Skill，检索调用时再次校验 KB 范围。电商与保险资产可以在不同 Workspace 管理，用户在自然语言或工具参数里指定其他 KB ID，也不能改变自己的授权。检索 SQL、资源查询和工具调用都要沿用正确范围。

如果增加缓存，我会把租户、授权范围和内容版本纳入键，并在读取时继续检查权限。验证会覆盖跨 Workspace ID、未绑定 KB、Skill 收窄后的调用和权限变化后的缓存读取。这样可以确认隔离贯穿发布、召回、上下文和输出，避免只在前端隐藏知识库入口。

1.8 审批、恢复、幂等与异步任务（Q49–Q58）

代码入口：审批中断装配、恢复执行、审批持久化、任务系统、审批安全测试。对应第 02、06、16、17 课及最终版补充实现。

Q49｜任务执行一半中断，怎样继续执行?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我会先确认任务为什么停下来、停在哪一步，以及外部操作是否已经生效，再决定怎样继续。按业务运行原因，我主要分六类：第一类是审批、补充资料等主动暂停；第二类是用户取消或连接断开；第三类是超时、迭代次数和额度耗尽；第四类是模型、网络或工具的临时故障；第五类是权限、安全或业务条件拒绝；第六类是进程崩溃、Worker 重启或租约失效。这是处理问题的分类方法，具体还需要结合系统记录判断。

识别时，我优先看结构化信号。审批暂停在 Eino 事件里有 `Action.Interrupted`，能够拿到 interrupt ID 和业务 Info，平台还有审批记录、checkpoint 及 `awaiting_approval` 状态。取消和截止时间可以用 `errors.Is` 判断 `context.Canceled`、`context.DeadlineExceeded`，再结合请求断开、显式取消记录或租约续期失败定位来源；单看 Canceled 无法确定就是用户点击了停止。模型和工具错误结合错误码、HTTP 状态、调用阶段与 Trace；进程突然退出通常没有正常错误返回，需要靠运行记录、心跳、租约和队列重投递发现。更完整的停止原因字段与任务状态是我会补充的治理能力。Go context 文档可以用于核对取消语义。

另外，从框架机制看，Eino 有基本 `Interrupt`、保存局部状态的 `StatefulInterrupt`，以及聚合子中断的 `CompositeInterrupt`。它们描述的是可恢复暂停的机制，和业务上的超时、取消、崩溃需要分别判断。我的项目用工具层 StatefulInterrupt 实现审批，保存审批信息和激活 Skill；框架 checkpoint 保存继续运行需要的执行状态。Eino 中断与恢复文档说明了这些接口。

对于第一类主动暂停，我的项目在敏感 Tool 真正执行之前创建审批并保存 checkpoint，同时绑定会话、固定版本和精确 interrupt ID。批准后，Worker 先原子领取该审批的执行权，再加载会话快照，用 `ResumeWithParams` 把批准数据交给指定断点。其他尚未批准的断点继续等待；如果后面又遇到敏感动作，还会产生新的审批。补资料与澄清也可以设计成同类暂停，但当前课程完整实现的是审批链路。

对于第二类取消或断线，我先按运行契约区分。当前在线请求依赖 context 时，取消要传播到模型和工具，用户明确取消后不应自动复活任务。持久化异步任务是否因前端断线而停止，需要独立政策。用户之后要求继续时，我会核对已完成操作和剩余步骤；已成功的退款不会随取消撤销。当前项目有请求取消传播，通用的取消后续跑产品能力还需要扩展。

对于第三类超时或预算停止，我会区分单次调用与整体任务限制。只读工具的局部超时可以在整体预算内有限重试；整体截止时间、最大迭代或额度已经用完，就应结束本轮并说明原因。需要继续时，重新确认授权与预算，加载可靠状态后再启动受控运行。当前项目有最大迭代和模型额度等约束，预算停止后的任意点自动恢复没有完整实现。

对于第四类临时故障，我会尽量重试失败步骤，并限制次数与等待时间。但写操作存在结果未知的情况：请求可能已被远端处理，只是响应丢失。这时先按业务单号查状态，或用同一个持久化幂等键重试；无法确认就转人工。仅凭本地报错重新执行整个 Agent，可能重复调拨、退款或赔付。

对于第五类权限、安全和业务拒绝，我会先解决原因。无权限需要合法授权，Guard 阻断要遵守策略，库存不足需要重新查询和规划；这些都不适合原参数无限重试。若恢复后操作对象、金额或其他已批准参数改变，应重新审批，执行时也要再校验当前业务状态。

对于第六类崩溃或执行权失效，我会先确认旧实例不能继续有效写回，再由新的执行者获取运行权，并核对 checkpoint、已完成步骤和外部结果。kbot 审批续跑有执行租约和 token 校验，续租失败会取消运行；不过当前 checkpoint 主要服务审批暂停，没有覆盖每个任意崩溃点。完整恢复还需持久化任务步骤、结果、重试策略和业务幂等，单靠保存聊天历史无法安全实现。

以项目中的库存调拨为例，我能演示审批前没有写入，批准后从目标断点执行，并验证重复批准和队列重复投递受到执行权控制；业务模拟器的幂等记录仍在内存中。接真实系统时，我会补持久化幂等和结果查询，并故障注入“业务已写入、平台未记成功”的窗口。恢复的目标是保留已发生的事实、继续未完成的工作，并保证副作用可核对。

Q50｜Checkpoint 应保存哪些内容，如何与聊天历史区分?

来源：**课程延伸题**。

参考回答：

我会从“恢复时需要知道什么”倒推 checkpoint 内容。至少要保留框架执行位置、运行状态、消息与工具调用上下文、等待中的中断及其局部状态；平台还要关联会话、审批、固定配置版本和目标 interrupt ID。实际序列化字段由所用框架定义，我不会用自己拼的聊天文本替代它。

项目由 Eino 生成框架 checkpoint，再包装格式版本、interrupt ID 和二进制数据，按审批及会话持久化。恢复时校验包装版本与必要字段，加载会话固定配置，再将批准数据发给目标断点。Skill 激活状态也需要恢复，防止续跑时权限范围发生变化。

聊天历史主要服务用户上下文，通常缺少精确执行位置和副作用状态。checkpoint 也不能单独证明远端业务结果，例如工具已写入后进程崩溃，还要查业务记录与幂等结果。我会测试损坏快照、错误会话、框架升级和恢复后再次中断，并管理敏感内容访问与保留期限。

Q51｜重复点击批准、队列重复投递时怎样防止重复副作用?

来源：**课程延伸题**。

参考回答：

我会分平台竞争和外部副作用两层处理重复。用户重复批准，先由审批状态机限制非法状态迁移；Worker 收到重复消息，执行前必须原子领取运行权。只有当前持有者可以续租和写回结果，旧执行者失去租约后需要停止。

kbot 的审批层有 BeginExecution、续租、execution token，以及完成和失败写回校验。它能降低两个 Worker 同时续跑同一审批的风险。但如果远端退款已经成功，本地还没写回就崩溃，新 Worker 仍可能再次发起请求，所以业务端还要用稳定幂等键、持久化记录和结果查询控制副作用。

当前业务模拟器会缓存重复键结果，但缓存位于进程内，重启后不保留。真实改造时我会补数据库唯一约束和请求指纹，测试并发批准、重复投递、租约过期、旧实例写回，以及远端成功后本地崩溃。我的目标是在明确条件下实现有效单次业务效果，不能只靠队列或锁就宣称端到端 exactly-once。

Q52｜幂等键怎样生成，同一键配不同参数怎么办?

来源：**课程延伸题**。

参考回答：

我会让幂等键绑定一次业务意图，同一次操作的重试复用该键。它可以由服务端在业务命令创建时生成并保存，也可以由明确的上游请求 ID 派生；关键是后续重试稳定，并且新操作不会误用旧键。范围需要包含租户、操作和资源等必要信息。

服务端保存键、规范化请求指纹、执行状态和结果。同键同参数返回原结果；同键异参返回冲突；处理中则按契约等待或查询，不能并发执行两次。幂等记录和业务变更最好在可协调的事务边界内处理，外部支付等场景还要利用对端幂等与查单能力。

课堂第 02 课的调拨服务有参数指纹冲突检测，最终业务模拟器主要按操作前缀和键缓存结果，持久化与指纹校验尚未普遍补齐。我会用同键同参、同键异参、并发请求和服务重启测试区分这些实现。模型可以传递已确定的键，稳定性应由受控业务流程保证，避免每次重试都生成一个新键。

Q53｜审批通过前后，库存或案件状态变化了怎么办?

来源：**课程延伸题**。

参考回答：

我会把批准的操作与执行时的业务条件分别校验。审批记录确认对象、动作和参数，等待期间库存、案件状态或可赔金额仍可能变化，所以恢复后不能直接沿用旧事实。执行接口需要读取当前状态，并在事务中检查关键条件。

例如调拨获批时源仓库存不足，业务服务应拒绝原数量；保险冻结获批时案件已经结案，也应返回状态冲突。若需要改变仓库、数量、对象或金额，我会重新生成方案并申请新的审批。更严格的方案可以在审批里保存对象版本，执行时用乐观锁检测变化。

课程业务服务已有库存、案件、部分金额和风险门禁，完整版本比对、审批过期和重新审批策略还需扩展。我会测试“申请后状态改变、批准后执行”的分支，确认失败原因、审批记录和实际业务结果一致。审批通过意味着允许尝试特定操作，最终成功仍由执行时的业务条件决定。

Q54｜拒绝审批、审批超时和参数修改怎样处理?

来源：**课程延伸题**。

参考回答：

我会先把审批决定和执行状态分开。拒绝表示这次绑定操作没有获得授权，应保留拒绝人、时间和原因，并停止对应敏感动作；超时属于审批生命周期策略，需要明确何时过期、是否通知或转人工。参数修改则要判断是否改变原来批准的业务意图。

项目通过受限审批卡片提交批准或拒绝，服务器校验角色、Workspace、会话归属及当前状态。一条自然语言"已经批准"不能替代这些校验。若调拨数量、仓库或赔付金额改变，我会创建新的审批记录，保留原审批与新请求之间的关联，避免旧批准覆盖新操作。

当前系统已有批准、拒绝和续跑链路，过期、委托、分级授权和会签需要按组织要求继续扩展。我会测试拒绝后重复批准、非审批人提交、修改参数、审批过期和并发决定等情况。页面反馈也要区分已批准、待执行、执行失败与业务完成，防止批准动作被误当成业务成功。

Q55｜用户等待太久要求停止，怎样取消 Agent?

来源：**公开面经题**，字节 AI 应用一面 S02。

参考回答：

我会先明确用户停止的是在线回答，还是一个已经持久化的业务任务。在线请求可以通过 context 传播取消，尽快停止模型流、HTTP 工具和后续调度；持久化任务则需要独立取消状态与接口。浏览器断线也需要按产品契约判断，不能直接推断所有后台任务都应停止。

项目的 SSE 使用请求上下文，Runtime 和下游工具接收 context，审批租约续期失败也会取消当前执行。我会检查所有可能阻塞的调用是否尊重取消，并确保流写入和资源清理能够退出。取消原因应记录清楚，避免把用户停止、超时和执行权丢失混为一类。

对于已经发生的副作用，context 取消无法自动撤销。比如退款请求已成功，就要保存并报告实际结果；响应未知则查单，必要时按业务补偿或人工流程处理。当前项目有基础取消传播，完整持久化任务取消与补偿仍需扩展，我会分别说明能力范围。

Q56｜怎样证明取消确实生效，而非界面停止显示?

来源：**公开面经题**，字节 AI 应用一面 S02。

参考回答：

我会从前端、Runtime、下游和业务结果四层验证取消。前端停止展示只是交互表现，服务端应收到取消信号，模型或工具调用应尽快退出，后续步骤也应停止。最后还要确认资源释放以及已发生操作的真实状态。

测试时我会使用可控制的慢模型和慢工具，在调用前、调用中、工具返回后等位置触发取消。检查下游是否收到 context done、取消后是否产生新的调用、连接与 goroutine 是否退出，以及审计和最终状态是否一致。对于忽略 context 的第三方接口，还要设计超时和清理兜底。

课程 SSE 与 Engine 测试可以作为这些验证的入口，我会按实际运行记录报告通过范围。第三方模型收到断开后是否停止推理和计费，需要向供应商核实并实测。写操作已经成功的分支尤其重要：系统应准确展示结果，再处理补偿，避免取消按钮让用户误以为业务从未执行。

Q57｜分布式锁、租约和执行 token 解决哪些问题?

来源：**公开面经题**，字节 AI 应用一面 S02、快手秋招 S07。

参考回答：

我会分别说明锁、租约和执行 token 的作用。锁或原子领取限制多个实例同时进入临界区；租约让故障实例的占有权能够过期；执行 token 用于识别当前持有者，防止旧实例在失去执行权后继续有效写回。它们解决的是并发协调的一部分。

kbot 的审批和会话轮次使用执行权领取、续租及 token 校验。Worker 续租失败会取消运行，完成和失败写回需要匹配当前 token。这样能限制重复消费以及旧实例覆盖新状态，但旧实例可能已经向外部业务发出了请求，平台 token 本身无法撤回远端副作用。

严格 fencing 还要求被保护资源识别并拒绝旧令牌，通常需要明确的顺序或版本语义；项目当前随机 token 的内部比较有它自己的适用范围。我会继续用业务幂等、对象版本和结果查询保护外部操作，测试租约过期后旧 Worker 恢复、网络分区和重复投递，确认内部状态与业务结果都可核对。

Q58｜哪些 Agent 任务适合异步，失败后如何处理?

来源：**公开面经题**，美团 Agent 一面 S06。

参考回答：

我会把耗时长、允许排队、不需要用户持续连接的工作放到异步任务，例如知识入库、批量评测和审批续跑。入口先确认任务可靠接收，再返回标识；用户通过状态或通知了解进度。短时交互仍可以同步或 SSE，避免所有任务都增加队列复杂度。

项目使用 Redis、Asynq 和独立 Worker，配置任务队列、超时和有界重试，并为审批投递提供补偿机制。消费端要按任务类型判断可重试性：解析临时失败可以重试，权限错误或业务拒绝应终止，写操作结果未知需要先核验。重复消费由执行权和业务幂等共同处理。

我还会管理重试耗尽后的失败记录、人工重放和过期任务。队列可靠性依赖 Redis 的持久化与运维配置，不能仅因用了队列就认为任务永远不会丢失。验收会包含入队失败、Worker 崩溃、重复投递、租约过期和人工重放，分别核对任务状态与实际副作用。

1.9 评测、观测、成本与安全（Q59–Q66）

代码入口：Eval、Judge、Trace、Guard、Langfuse 课件。对应第 18–20 课。

Q59｜Agent 成功率怎么计算，怎样证明数字可信?

来源：**公开面经题**，阿里 AI 全栈一面 S03、4399 Agent 面经 S09。

参考回答：

我会先定义成功标准，再计算成功率。Agent 的成功至少包含答案或建议正确、工具与参数合理、关键业务状态达到预期，以及审批和权限约束没有违反。统计时固定任务集合、模型、Prompt、工具版本、运行次数和环境，再用成功任务数除以总任务数。

我也会分开报告错误类型和安全失败。比如订单回答措辞不完整、工具参数错误和未审批写入，影响程度不同；总体平均数可能掩盖后者。带随机性的真实模型需要重复运行，并说明样本数量与波动。每次评测最好保留逐例输入、轨迹、结果和判定依据，便于复查。

课程跨境和保险业务样本分别只有 3 条、4 条，受控 E2E 主要证明特定链路能够运行。我会把这些结果表述为验收案例，真实泛化成功率需要扩充测试集并接真实模型复测。没有完成测量时，我会说明验证范围，不填一个缺少依据的准确率或业务收益。

Q60｜评测集如何构建，怎样避免训练或调 Prompt 时污染?

来源：**公开面经题**，阿里 AI 全栈一面 S03。

参考回答：

我构建评测集会先列业务能力和风险维度，再选择案例。来源可以是经过授权和审核的真实任务，以及为边界条件设计的合成样本；覆盖正常、资料缺失、错误前提、越权、工具故障、重复提交和状态变化。每条都要明确预期结果及判定依据。

项目 JSONL 可以作为种子，我会扩充电商多币种、SLA 和库存边界，以及保险期间、免赔额、补件和高风险分流。除了期望答案，还记录允许或禁止的工具、关键规则和业务最终状态；业务人员复核样本，避免评测本身把错误规则固化下来。

防污染时，我会按来源、时间或业务对象划分开发集与保留测试集，避免同一个模板轻微改写后分散到两边。Prompt 调优只使用开发集，最后用保留集检查泛化，并保存数据集版本和变更记录。若测试集已经被反复用来选方案，我会重新补充独立样本，明确当前数字的适用范围。

Q61｜LLM-as-a-Judge 的优点、偏差与适用范围是什么？

来源：**课程延伸题**。

参考回答：

我会用 LLM Judge 评估解释的语义一致性、完整性和证据使用情况，适合人工难以逐条检查的大量开放回答。但它的评分可能受表达长度、顺序、模型偏好和输入中的恶意指令影响，也可能多次评分不一致，所以需要固定评审规则并抽样人工复核。

项目提供确定性 Judge 和 light/full LLM Judge，后者返回分数与理由。保险可赔金额是否为规则结果、冻结是否生效、审批前有没有写入，这些适合用确定性断言；解释是否覆盖关键依据，可以交给语义评审。若两类判定冲突，我会先查原始证据和业务状态。

当前 Judge 主要比较文本，完整工具轨迹与业务状态评分需要继续接入。扩展时我会准备人工标注对照集，检查评审模型与人的一致性、重复评分波动和已知错误漏判。资金或权限相关断言不会仅靠一个模型分数决定，评审理由也需要支持回溯。

Q62｜如何设计 Eval 发布门禁和回滚？

来源：**课程延伸题**。

参考回答：

我会把候选版本、评测数据集、模型与工具依赖、判定规则一起固定下来，再设置发布门槛。门槛既包括总体质量，也包括关键安全分项，例如未授权检索或未审批写入不能被其他高分案例抵消。评测结果还要能追到逐例证据。

kbot 已有 Eval 和版本发布约束，更细的安全分项门禁可以继续扩展。发布时保留上一稳定版本的环境指针，新会话使用候选版本，旧会话仍按原快照运行。若新版本失败增加，我会回滚后续流量，并核查等待审批的任务和版本兼容情况。

配置回滚解决的是后续行为，已经发生的退款、赔付或冻结还需单独查询和处理。我的验收会包含评测未达标禁止发布、稳定版本切回、旧会话不漂移和已发生操作的结果核对。回滚是否有效，要同时看版本指向、请求行为和业务状态。

Q63｜Agent 能否被观测，应该记录哪些节点？

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

Agent 可以观测，但我会区分可见执行过程与模型内部推理。可观测的包括输入输出、模型调用、检索候选、工具参数与结果、审批暂停和恢复、错误及耗时；模型未公开的内部推理不能从这些记录中完整还原。

项目使用 OpenTelemetry 和 Langfuse，并在控制台提供 Trace 深链。我会关联 Workspace、用户、Conversation ID、Agent 与资产版本、调用次数、Token 和错误类型。一次任务拆成不同 Span 后，可以判断耗时集中在模型、检索、外部工具还是审批等待；异步恢复还要通过审批和会话标识关联原始请求。

排查订单诊断时，我会先看工具调用顺序和结果，再看模型如何使用这些事实，最后核对业务状态。采集内容受隐私与权限策略控制，敏感材料不应无条件进入日志。观测能力的价值是提供可核验的证据和瓶颈定位，我会用一次正常任务及一次失败任务展示这点。

Q64｜日志怎样定位 Agent 卡在哪里，审计与 Trace 如何配合?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我会先用 Conversation ID 或 Trace ID 找到最后完成的阶段，再区分它在等待什么。最后一个节点是模型请求，就查供应商响应、超时和额度；停在工具节点，就看外部接口、参数和业务拒绝；等待审批则查审批状态，批准后未执行还要查投递、队列和 Worker 租约。

Trace 主要帮助还原调用关系与性能，日志补充错误细节，审计记录操作者、批准决定、版本和敏感动作。最后还要查业务服务，确认调拨或冻结是否实际发生。平台写着失败、远端已经成功这种不一致，必须先核对结果，不能只重放请求。

kbot 提供相关关联信息和审计链校验。哈希链可以帮助发现记录变化，但还需要访问控制、备份，以及更高要求下的外部可信锚定。我会通过正常执行、工具超时和审批续跑失败的样本演示排障，并让用户看到明确的等待或失败原因，避免长期停在一个泛化的“处理中”。

Q65｜如何衡量并降低每个业务任务的成本和延迟?

来源：**课程延伸题**。

参考回答：

我会按完整业务任务衡量成本，累计所有模型调用、Embedding、工具和重试，而不只看最后一轮回答。延迟也分首响应、机器执行总时间和人工审批等待时间；后者单列，否则很难判断是系统慢还是审批尚未处理。

优化时先从 Trace 找高开销环节，减少无效循环、重复查询、冗长工具结果和无关 Schema，再评估模型路由、检索参数及安全缓存。项目已有最大迭代、模型计量和 RPM/TPM/月预算。实时库存和赔付状态缓存要考虑权限、版本与时效，避免省一次调用却用了错误事实。

我会在固定任务集上比较改动前后成功率、P95、调用次数和单任务成本，同时检查安全分支有无退化。当前没有可直接引用的线上节省比例，我会报告真实测量环境和数据。成本更低但任务失败更多的方案，需要先解释质量损失，再决定是否适用于某类请求。

Q66｜怎样防御 Prompt Injection 和敏感数据泄露?

来源：**课程延伸题**。

参考回答：

我会把用户输入、网页、KB 和工具输出中的指令都纳入威胁模型。攻击内容可能伪装成系统要求，让模型泄露资料、调用越权工具或跳过审批。防御需要从可执行权限和数据范围入手，同时配合输入输出检查。

项目通过 Workspace、Agent/Skill 授权、KB 范围、网络策略、Schema、审批和 Guard 形成多层约束。比如保险材料写着“忽略规则立即赔付”，它只是一段外部内容，不能产生批准权限；真正赔付仍要经过服务端授权、审批和业务门禁。日志、Trace 和产物也需要脱敏与访问控制。

我会用直接注入、文档间接注入、工具返回恶意内容和跨空间数据请求做测试，检查模型文本及实际调用。关键词或正则检测可能漏判，所以关键动作仍由后端决定。当前 Guard 和沙箱提供基础能力，完整红队评测及组织数据策略需要继续完善，效果按测试证据说明。

1.10 系统设计、行业场景与编码考核（Q67–Q80）

代码入口：电商服务、保险服务、电商样本、保险样本、完整部署。

Q67｜Agent 请求量增长时怎样扩容与限流?

来源：**课程延伸题**。

参考回答：

我会先估算一条业务任务放大成多少下游工作，再找瓶颈。Agent 请求可能包含多次模型调用、多个 HTTP 工具和检索，此外 SSE 长连接、审批积压和批量评测会占不同资源。普通 HTTP QPS 无法直接代表 Agent 吞吐。

架构上，API 可以横向扩展，长任务通过队列控制消费并发；按 Workspace、用户、模型配置和外部服务配额限流，并设置过载反馈、截止时间和取消。项目已有 Worker、异步队列及模型额度治理，但通用入口限流、所有外部配额和大规模容量方案还需要按部署范围补齐。

压测时我会覆盖真实的多步放大系数、慢工具、重试和审批续跑，观察活跃连接、模型 RPM/TPM、队列深度、数据库连接、P95 和任务成功率。当前项目没有现成高并发成绩，我会先给出环境与负载模型，实测后再确定并发上限和扩容策略。

Q68｜Workspace 隔离如何从接口延伸到数据与缓存?

来源：**课程延伸题**。

参考回答：

我会把 Workspace 当成贯穿系统的授权范围。接口层先验证用户成员关系和角色，读取或修改资源时同时检查 workspace_id；跨资源引用也要确认属于同一范围。前端切换空间可以方便操作，真正授权由服务端和存储约束落实。

项目有五档角色、Workspace 中间件、Runtime 归属校验和部分数据库复合外键。Agent、Tool、Skill、KB 的发布与绑定也要校验空间。电商和保险可以分别安装到独立 Workspace 或环境，避免资产混用；内部 SDK 调用同样需要保留身份和范围。

我还会检查缓存、队列、对象存储和 Trace 是否延续这个边界。新增缓存键包含范围，Worker 不能只拿到资源 ID 就绕过归属核验，日志查询也需控制访问。验证会覆盖猜测其他空间 ID、跨空间绑定、审批越权和缓存串读，并区分已实现约束与需要补齐的路径。

Q69｜平台持久化与业务持久化怎样区分，为什么 Mock E2E 不够?

来源：**课程延伸题**。

参考回答：

我会先说明两套状态由谁负责。平台持久化保存配置版本、会话、审批、checkpoint 和评测；业务持久化保存订单、库存、赔付记录和幂等结果。平台数据库正常，并不能保证业务服务重启后仍记得一次退款是否执行。

课程最终版的平台使用 PostgreSQL Store，两个行业模拟器的业务状态与幂等结果主要保存在内存 map。Mock E2E 能检查受控工具序列、审批前无写入和批准后恢复，但无法证明真实模型会正确选择工具，也无法验证业务重启后的幂等或第三方资金通道可靠性。

接真实系统时，我会增加契约测试、持久化业务状态、请求指纹和结果查询，并故障注入"业务成功、平台写回失败"等窗口。验收记录分别列出 Mock 流程、真实模型表现和业务可靠性，避免用同一份通过结果覆盖三个不同问题。

Q70｜从课程 Compose 环境到实际部署，还需要完成哪些工作?

来源：**课程延伸题**。

参考回答：

我会从身份、数据、网络、可靠性和运维几个方面补齐部署要求。身份侧接 SSO/OIDC 和组织角色；数据侧管理权限、保留期限与备份恢复；网络和凭据侧使用 TLS、受控出口及外部密钥管理。具体要求要结合部署组织与业务环境确定。

课程已有 Docker Compose、Worker、Sandbox Runner 和 CI，适合本地复现核心链路。实际部署还要确认 PostgreSQL 与 Redis 的高可用、持久化、恢复演练、监控告警和容量，隔离 Runner 控制权限，并接入真实业务契约与持久化幂等。每项都应有失败时的处理责任和验证记录。

开放能力时，我会先用只读或影子验证检查质量，再逐步开放带审批的写操作。电商和保险分别验收数据授权、状态门禁、异常核对和人工处理机制。当前项目没有真实生产规模和业务收益数据，我会把本地成果、部署改造和待验证效果分别讲清楚。

Q71｜海外电商订单缺货且即将超时，Agent 应怎样处置?

来源：**课程延伸题**。

参考回答：

我会先确认订单、SKU、数量、发货截止和当前状态，再查询各仓库存与物流方案。以课程 TTS-20260801-1001 为例，深圳仓可用量为 0，洛杉矶仓为 18；这些是方案依据，还需要结合调拨周期、运输时间、截单和成本判断可行性。

如果选择跨仓调拨，我会明确源仓、目标仓、SKU、数量和稳定幂等键，准备审批请求；批准后调用 create_inventory_transfer，由业务服务重新检查库存和条件。回答会区分建议方案、等待审批、调拨创建成功，以及实际履约进度，避免把创建记录直接说成订单已发货。

当前项目的物流和库存来自模拟服务，SLA 标志无法单独证明真实调拨加运输一定按时。真实接入需要补 OMS/WMS、时区和实际路线数据。如果无法满足截止时间，我会说明缺口，查询符合条件的替代方案或退款资格，再交授权人员决定。演示时我会同时展示正常调拨链路和库存变化导致的拒绝分支。

Q72｜订单结算对账怎样做到金额正确且可追溯?

来源：**公开面经题**，京东后端实习 S08。

参考回答：

我会先固定对账范围，包括商户、账期、币种、订单集合和费用口径，再从业务接口读取应结、实结与明细。差异由确定性逻辑计算，模型负责整理原因和证据；存在口径或币种不一致时，我会先澄清，避免直接相减。

课程 STMT-2026-31 的应结为 118.47 USD、实结为 106.95 USD，差异是 11.52 USD。我会展示来源和计算过程，再用结算 Skill 准备申诉材料。create_reconciliation_case 需要审批，创建成功只说明申诉记录已建立，后续退款或财务入账状态需要独立查询。

模拟器使用 float64，正式改造时我会采用最小货币单位整数或定点十进制，固定舍入、汇率版本和费用规则，并保存账单快照与请求标识。验证覆盖小数边界、重复明细、多币种、重复申诉和响应丢失。金额正确性与可追溯性都需要业务记录支持，模型文本无法替代账本核对。

Q73｜保险案件的条款检索、责任判断与赔款计算怎样分工?

来源：**课程延伸题**。

参考回答：

我会按事实、规则证据、计算和执行四步处理。先用 get_claim 和 get_policy 读取案件、保单、事故时间和索赔信息；再从 KB 查适用条款及审核 SOP，确认版本与条件；随后调用 evaluate_coverage 获取结构化责任理由和可赔上限。

课程 CLM-2026-0001 索赔 6800 CNY，模拟限额为 20000、免赔额为 500，规则 `max(0, min(6800, 20000)-500)` 得到 6300 CNY。我会依据接口结果解释计算过程，区分规则评估结果和实际批准金额。赔付动作还需人工审批，执行时再次核验案件状态与金额。

这套公式对应本项目模拟产品，真实条款还会有责任范围、例外、比例和其他适用规则，需要业务审核后实现为可版本化规则。模型不自行创造赔款公式。我会用保险期间边界、金额超限、缺少资料和审批后状态变化测试链路，并展示条款来源、理由码、计算结果和执行记录。

Q74｜高风险保险案件怎样处理，如何避免把风险信号当成欺诈定论?

来源：**课程延伸题**。

参考回答：

我会把责任判断、风险线索和处置决定分别表达。风险分高说明需要进一步核验，不能直接给出欺诈定论。输出中保留每个信号的来源、时间和待核实事项，由有权限的人员依据证据作最终判断。

课程 CLM-2026-0002 的事故时间早于保单生效，模拟风险分为 0.92，并带重复影像信号；按项目 0.70 阈值进入人工处置。我会核对保单和案件事实，说明责任问题及风险线索；支付冻结和调查立案分别使用敏感工具审批，并检查各自前置状态，避免自动赔付或越序处置。

这些风险分和影像信号来自模拟器预置数据，项目没有实现视觉查重或学习型反欺诈模型。真实接入时，我会补信号服务契约、材料权限、误报处理和人工复核记录。验收会检查高风险禁止自动赔付、审批前无冻结写入、批准后的状态变化，以及解释中是否保留不确定性。

Q75｜如果要求支持 OCR、图片和语音，怎样扩展当前系统?

来源：**公开面经题**，字节 AI 全栈一面 S04中的多模态实现考点。

参考回答：

我会先设计文件进入系统的受控链路。上传时核验类型、大小、来源和权限，保存原文件及访问范围；然后按任务调用 OCR、ASR 或视觉服务，把提取结果转成可追溯的文本和结构化字段。页码、时间戳、置信度和处理版本都要保留，方便复核。

保险材料里，票据金额、日期、保单号等关键字段需要与原图关联，低置信度或冲突转人工；电商语音指令可以先转写再确认订单对象。图片相似性和文字识别是不同能力，需要各自的模型、接口和评测，不能只靠一个附件字段证明已经支持。

当前行业项目主要使用文本、结构化接口和预置信号，多模态处理是扩展方案。我会先把识别结果接到现有工具和证据链，再测试错字、模糊图、重复上传、恶意文件和越权读取。关键副作用继续走原有授权与审批，避免识别错误直接触发业务操作。

Q76｜使用 AI Coding 如何证明自己真正掌握项目?

来源：**公开面经题**，字节 AI 全栈一面 S04、4399 Agent 面经 S09。

参考回答：

我使用 AI Coding 时，会先自己明确接口契约、状态变化和验收条件，再让工具辅助生成或修改代码。生成后由我检查依赖、授权、并发、错误处理和测试是否覆盖真正的风险。能编译只是基础，关键是我能解释它为什么正确，以及失败时会发生什么。

对这套项目，我可以沿一次订单请求解释入口、模型、工具、审批和业务结果，也可以展开重复审批、同键异参、跨空间 KB 和取消慢工具这些分支。现场修改一个小功能时，我会先分析受影响的契约和用例，再实施并验证，而不会只靠继续生成来碰结果。

个人贡献我会按实际提交和实验说明，区分复现课程能力、独立实现和二次开发。AI 生成比例无法直接代表掌握程度，我愿意用代码讲解、故障分析和现场修改证明自己能维护相应模块。尚未亲自完成的功能会明确说明，避免把平台全部能力都算作个人成果。

Q77｜用两个栈实现线程安全队列，如何联系 Agent 工程?

来源：**课程延伸题**。

参考回答：

我会维护入栈 A 和出栈 B。入队时直接压入 A；出队时，如果 B 不为空，就弹出 B；如果 B 为空，再把 A 的全部元素逐个弹出压入 B，此时最早入队的元素就位于 B 顶。只有 B 空时才搬运，才能维持先进先出。

复杂度方面，每个元素最多经历一次 A 到 B 的搬运，入队和出队均摊 O(1)，单次触发搬运的出队最坏 O(n)，空间 O(n)。线程安全时，我会用同一把互斥锁覆盖检查、搬运和弹出整个操作；分别给两个栈加锁却缺少整体协议，可能破坏状态一致性。空队列我会按约定返回明确的错误或状态。

测试会覆盖空队列、交替入出、多次搬运和并发消费者；如果需要阻塞等待，还要增加条件变量或合适的 channel 设计。它适合解释单进程队列，而课程审批续跑使用 Asynq，还涉及持久化、多进程消费和重试，这些需要更完整的任务系统。

Q78｜二叉树序列化与反序列化怎样写得可验证?

来源：**公开面经题**，百度 Agent 实习 S01。

参考回答：

我可以用前序遍历实现。序列化时依次记录节点值、左子树和右子树，空节点也写明确标记，例如 #；节点值和标记采用无歧义分隔。空标记保留了结构信息，重复值的树也能恢复，只有节点值序列通常不够。

反序列化时按相同顺序消费输入：读到 # 返回空，读到合法值就创建节点，再递归构造左右子树。需要检查输入提前结束、非法值和多余尾部内容，不能只对正常数据工作。时间和输出空间是 O(n)，递归辅助栈 O(h)，退化链状树时 h 可以达到 n。

我会测试空树、单节点、偏斜树、负数、重复值和无效输入，再验证序列化往返后结构与数值都一致。与项目 checkpoint 的联系是必须保存足够状态并定义格式边界；ADK 快照还涉及框架版本和中断状态，需要兼容性校验。树算法本身并不等于已经实现 Agent 的通用恢复。

Q79｜删除链表倒数第 k 个节点，面试中要讲哪些边界?

来源：**公开面经题**，美团 Keeta 一面 S05。

参考回答：

我会先约定 k 从 1 开始，空链表或 k 越界返回明确错误，并用虚拟头节点统一删除头节点的情况。快慢指针都从虚拟头开始，快指针先走 k 步；过程中到空说明 k 大于链表长度，需要停止处理。

然后让两个指针同步移动，直到快指针的 next 为空。此时慢指针指向待删除节点的前驱，执行 `slow.next = slow.next.next` 即可。比如长度为 5、k=2，慢指针最后停在第 3 个节点，删除第 4 个。时间 O(n)，辅助空间 O(1)。

我会检查 k≤0、k 大于长度、删除头尾、单节点和空链表，测试返回头节点是否正确。这个算法考察指针和边界处理，我能独立实现并说明循环不变量；它属于编码基础练习，与 kbot 业务功能分别表述。

Q80｜如何回答“最大难点、失败案例，以及下一步改进”?

来源：**公开面经题**，阿里 AI 全栈一面 S03、vivo Agent 面经 S10。

参考回答：

这套项目里，我认为最难的部分是把敏感操作的暂停、批准、恢复和重复执行控制连起来。我需要同时理解框架中断状态、平台审批状态和外部业务结果，任何一层只看本地成功或失败，都可能误判剩余工作。

我会用故障模拟说明这个难点：调拨请求已在业务服务成功，Worker 在写回结果前退出，重投递后平台需要判断它是否还要再次执行。我的分析路径是先关联审批、工具调用与 Trace，再查询业务结果，检查固定版本、精确断点、执行租约和幂等键分别保护什么。项目已有目标断点恢复及审批执行权控制，业务模拟器的幂等记录仍在内存中。

下一步我会优先补持久化幂等、同键异参检测和业务结果查询，再用重启、重复投递及写回失败测试证明效果。如果我没有亲历线上事故，就会明确这是复现或故障模拟。这样我能讲清问题、方案、验证和剩余边界，同时让个人贡献与实际证据一致。

1.11 学员实面专项：七道问题与知识点详解（Q81–Q87）

本节来源为 **U01：课程方提供的学员实际面试反馈**，收录日期为 2026-10-11，实际面试日期及企业未提供。保留原始问法的考察意图，按语境将“Promet”规范为 **Prompt**，将“React”规范为 **ReAct**。下文均为课程化参考答案，学员应按个人实际完成的模块调整“我实现了”的表述。

每题按“口述回答—知识点—项目落地—追问”准备。先按完整逻辑演练，再练习在有限时间内保留定义、判断依据和项目方案；面试官追问时继续展开代码、异常分支与验证证据。

学员反馈原题	专项题号	原题库关联	重点知识
向量化了解过吗，介绍一下	Q81	Q37–Q43	Embedding、相似度、索引、混合检索与评测
对于 Promet 优化有什么自己的思路方法	Q82	Q14、Q18、Q59–Q62	失败归因、Prompt 结构、示例、版本及对照实验
编排方式了解过吗，说一下 React 和 multi-agent	Q83	Q02–Q09	单 Agent 循环、角色协作、Supervisor、Pipeline
说一下 Skill 封装调用的方法	Q84	Q25、Q26、Q30、Q31	SKILL.md、发布绑定、渐进加载、触发与授权
沙盒隔离的方法	Q85	Q36	namespaces、cgroups、权限、网络及容器生命周期
说一下 A2A 协议	Q86	Q32–Q34	AgentCard、Message、Task、Artifact、协议绑定
怎么实现把 Agent 嵌入项目中	Q87	Q01、Q10、Q49、Q68、Q69	业务入口、身份映射、Runtime、Tool、审批与结果回写

Q81｜向量化了解过吗？结合你的项目介绍一下

来源：**学员实面题**，U01 第 1 题。

参考回答：

了解，我在项目的知识检索里用到了向量化。我的理解是，把文本等数据转成可计算的数值表示；语义检索通常用 Embedding 模型把问题和文档片段映射到兼容的向量空间，再通过相似度寻找相关内容。词袋、TF-IDF、哈希也属于数值表示方法，具体效果取决于方法和任务。

入库时，我先把履约 SOP 或保险条款切成片段，生成向量并保存来源；查询时生成问题向量，在授权 KB 中召回，再结合关键词结果通过 RRF 融合。pgvector 的 `<=>` 表示余弦距离，越小越接近；单位归一化后，点积与余弦相似度相等。向量模型负责生成表示，数据库负责存储和搜索，HNSW 等索引负责加速，这几层需要分别理解。

项目有 Embedder 抽象和真实服务适配，本地 LocalEmbedder 是词袋哈希加 L2 归一化，主要支撑离线链路，无法据此证明真实语义质量。选择或更换模型时，我会核对维度、预处理和向量空间兼容性，必要时重建文档向量，并比较 Recall、答案质量和延迟。订单号、SKU 适合精确匹配，实时库存还需要业务接口；语义相近的条款也要继续核验版本和适用条件。

需要掌握的知识点：

知识点	面试时应说明的内容
向量化与 Embedding	数值表示有多种方法，包括词袋、TF-IDF、哈希及训练得到的稠密向量；语义检索通常采用适合检索任务的 Embedding
模型与向量空间	文档和查询使用兼容的模型、预处理及查询/文档编码方式；维度相同不足以证明向量空间兼容
相似度	`cos(q,d)=q·d/(‖q‖‖d‖)`；单位归一化后点积等于余弦相似度；pgvector 的 `<=>` 表示余弦距离，距离越小越接近
向量检索与索引	Embedding 生成表示，向量库保存和搜索表示，HNSW 等索引加速近似搜索；三者各有职责
语义与事实	相似度表示相关性，无法直接证明条款适用或结论正确；还需检查时间、版本、权限和实际证据
更新与迁移	文档更新后重新切分、向量化和索引；更换模型时按新向量空间重建并验证，保留旧索引以便切换
效果验证	用有标注的业务查询比较 Recall@K、排序质量、答案正确率、P95 与成本，避免仅用几句近义句演示下结论

课程项目落地： `Embedder` 提供 `Embed` 与 `Dim` 抽象；LocalEmbedder 用词袋哈希及 L2 归一化支撑离线链路，真实服务适配器检查返回条数与维度。持久化检索器结合全文排序、余弦距离和 RRF。保险条款需要匹配产品版本与事故时间，库存数量则从业务接口获取。模型选型与参数以实测说明，不虚构检索提升。

追问准备：

- **“为什么还要关键词检索？”** SKU、条款编号和专有名词需要精确匹配；混合检索通过不同信号补充候选。
- **“向量维度越大越好吗？”** 需要同时比较检索质量、模型能力、存储、带宽和延迟，维度本身不能保证效果。
- **“换模型只改查询端可以吗？”** 应核对模型兼容性；向量空间变化时文档端也要重建，并重新建立评测基线。

代码与资料：Embedding 抽象和离线实现、真实服务适配、pgvector 检索、语义相似度文档 T16、pgvector T07。

Q82｜对于 Prompt 优化，你有什么自己的思路和方法?

来源：**学员实面题**，U01 第 2 题；反馈中原拼写为“Promet”。

参考回答：

我的方法是先定义成功标准和基线，再用失败样本定位问题，最后通过固定评测验证改动。我会先看工具轨迹与实际业务结果，区分提示不清、检索缺证据、工具错误、上下文丢失和模型能力不足。原因不同，修复位置也不同。

确认属于 Prompt 问题后，我会把角色、目标、事实来源、工具用途、操作边界、异常处理和输出要求写清楚，再加入少量覆盖典型失败的示例。比如模型把等待退款审批说成已退款，我会核对它收到的状态，并要求回答区分建议、待审批、已执行和结果未知；保险回答则围绕保单与事故时间、规则理由码、可赔上限、风险及待核验项组织。

项目把 System Prompt 与带变量 Schema 的 User Prompt Template 分开管理，由 Agent 版本固定引用。每次调优尽量只改一个因素，固定模型与工具，用开发集分析逐例结果，再用保留测试集检查泛化；随机性明显时重复运行。比较成功率、关键违规、Token 和延迟后再发布，并保留回滚版本。尚未测量的收益我不会写成提升比例，能够展示的应该是失败样本、版本差异和实验记录。

可复用的方法：

1. **定目标和基线。** 明确任务输入、预期工具、答案证据、业务状态与禁止行为，保存当前 Prompt、模型及资产版本。
2. **先归因。** 区分提示不清、检索漏召回、工具参数或结果错误、上下文缺失及模型能力限制，用 Trace 和业务结果定位。
3. **整理 Prompt 结构。** 角色与职责、目标、可信数据来源、工具用途、状态约束、异常分支及输出要求分别写清，减少重复或冲突指令。
4. **补少量有效示例。** 选择金额边界、资料缺失、审批拒绝和工具失败等代表案例；示例与真实字段、工具名称保持一致。
5. **采用结构化契约。** 用户任务变量通过 JSON Schema 校验，工具参数与副作用继续由后端验证；回答中区分事实、建议、待审批及已执行结果。
6. **小步对照与消融。** 尽量一次改变一个因素，固定模型参数与任务集合；随机性较大时重复运行，单独保留未用于调优的测试集。
7. **版本化与回归。** 记录变更动机及逐例结果，检查关键安全用例、成本和延迟，发布后监控并保留回滚入口。

课程项目示例： 电商 Prompt 可明确“查询订单、库存和物流事实后提出方案；证据不足时说明缺失项；调拨与退款进入审批，只有成功工具结果才能作为已执行依据”。保险 Prompt 可要求输出“保单版本与事故时间—责任理由码—规则接口返回的可赔上限—风险信号与待核验项—审批状态”。这是讲解用的约束示例，实际优化成果需要附个人实验记录。

追问准备：

- **“具体优化过什么，提升多少？”** 选自己的一个失败样本，展示改动前后 Prompt、固定测试集、运行次数与结果；尚未测量时只说明改动和验收范围。
- **“加一句禁止越权就足够吗？”** Prompt 说明边界，服务端权限、工具门禁和审批执行约束，安全测试验证实际效果。
- **“什么时候不继续改 Prompt？”** 失败由知识缺失、接口错误或模型能力造成时，分别修数据、工具或评估模型方案，随后回归验证。

代码与资料：Prompt 服务、课程 Prompt 预设、供应链任务模板、Eval 服务。

Q83｜了解哪些编排方式？说一下 ReAct 和 Multi-Agent

来源：**学员实面题**，U01 第 3 题；反馈中“React”按 Agent 语境整理为“ReAct”。

参考回答：

我了解 ReAct、Plan-and-Execute、Workflow/Pipeline，以及 Supervisor 等多 Agent 协作方式。我会先分清层次：ReAct 说明一个 Agent 如何根据模型决策与工具观察循环行动，Multi-Agent 说明多个 Agent 如何分工；两者可以组合，专家内部和主管都可以运行 ReAct。

我的项目主循环用 Eino ADK，输入任务和允许工具，模型选择调用，执行层校验并返回结果，再进入下一轮，直到回答、报错、取消、达到限制或审批暂停。订单缺货诊断适合这种动态查询。团队层提供 Supervisor 与顺序 Pipeline，前者通过 AgentTool 选择固定版本成员并汇总，后者按阶段传递输出；当前工具执行采用顺序策略。

我会按依赖和风险选择编排。独立查询未来可以并行，强依赖步骤保持顺序，责任计算和写入前置条件放在业务服务。多 Agent 还要定义交接契约、证据来源、超时和冲突处理，不能只把角色名称拆开。我会与单 Agent 基线比较成功率、成本和延迟；通用规划器、并行 DAG 和无进展检测是当前可以继续扩展的部分。

需要掌握的知识点：

方式或概念	主要解决的问题	本课程的例子与范围
ReAct	根据观察结果动态决定下一步行动	订单→库存→物流查询；模型可补充检索，再提出处理方案
Plan-and-Execute	先形成计划，再逐步执行与必要重规划	通用规划器可作为扩展；当前主要链路采用 ReAct
Workflow / Pipeline	固定步骤、依赖与输入输出传递	Pipeline 将上一成员输出传给下一成员；业务服务校验硬性前置状态
Multi-Agent	按职责、知识及权限拆分协作成员	商品运营与供应链、理赔审核与反欺诈可按职责设计协作
Supervisor	在多个成员之间动态分派并整合结果	主管通过 AgentTool 调用固定版本的成员
并行与汇总	同时处理独立任务并管理部分失败	当前 Supervisor 工具顺序执行；通用并行 DAG 调度需要新增实现

讲清一次执行： 用户要求处理缺货订单，ReAct 先查询订单，读取库存后再决定是否查询物流或 SOP；需要创建调拨时进入审批。采用团队方案时，主管可把履约信息整理交给商品运营角色、把恢复方案交给供应链角色；角色之间的输入、输出和最终业务权限仍需明确。模型产生工具调用和可见事件即可说明执行过程，无需声称能够读取完整内部思维链。

追问准备：

- **“多个 Agent 是否一定更好？”** 用同一业务评测集比较单 Agent 与团队方案，关注任务成功率、证据损失、调用成本及完成时间。
- **“怎么防止无限循环？”** 限制最大迭代、重试与整体预算；进一步检测重复调用或无进展状态，达到阈值时结束或转人工。课程已有限步数，无进展检测可扩展。
- **“结果冲突怎么办？”** 保留每个成员的证据与版本，关键事实重新查询权威接口；无法消解时显式输出争议并交人工确认。

代码与资料：ADK 主循环、Supervisor、Pipeline、ReAct 论文 T02、编排模式 T01。

Q84｜说一下 Skill 封装与调用的完整方法

来源：**学员实面题**，U01 第 4 题。

参考回答：

我会把一类可复用业务任务封装成 Skill，SKILL.md 头部描述名称、触发条件、工具与 KB 范围、网络需求，正文说明输入、前置核验、操作步骤、异常分支和结果要求。比如订单异常恢复，需要先查订单、库存和物流，核验证据，再准备可审批的处理方案。

平台解析后创建草稿版本，发布时校验名称、描述、正文长度，以及同 Workspace 的 Tool/KB 引用；已发布版本再绑定到 Agent 快照。运行时先通过 Backend.List 提供简短描述，选中后通过 skill 工具和 Backend.Get 加载正文；用户也可以用 `/skill order_exception_recovery` 或 `/order_exception_recovery` 显式选择。

我重点检查加载后的授权。Middleware 校验绑定与网络需求，记录触发来源和版本，激活后收窄工具信息；真正调用时再检查 Tool、KB 和网络。敏感调拨仍走审批，恢复时保留激活 Skill 的约束。disable-model-invocation 控制模型自主选择，显式调用仍受授权限制。Skill 的流程说明需要评测，业务前置条件与副作用控制继续由执行层保证。

封装与调用步骤：

1. **确定任务边界。** 封装一类稳定复用的任务，如订单异常恢复、结算对账或理赔责任审核，明确输入、成功条件和失败分支。
2. **编写 SKILL.md。** 课程解析器识别 `name`、`description`、`allowed-tools`、`allowed-kbs`、`disable-model-invocation`、`requires_network`；正文写清流程与证据要求。字段属于 kbot 的实现契约，其他框架需要按其 Schema 适配。
3. **创建与发布版本。** 创建草稿，校验名称、描述、正文长度及同 Workspace 的 Tool/KB 引用，完成相关测试后发布。
4. **绑定 Agent。** 将已发布 SkillVersion 纳入 Agent 快照，绑定其依赖的 Tool 与 KB；执行权限受两层授权共同限制。
5. **触发并渐进加载。** Middleware 先通过 Backend.List 提供简短元信息，选中后通过 Backend.Get 读取正文；显式入口支持 `/skill order_exception_recovery` 或 `/order_exception_recovery`。
6. **执行与留痕。** 记录 Skill 触发来源和版本，收窄工具信息，调用前再次检查权限；敏感工具触发审批，恢复时继续保留激活 Skill 的约束。

课程订单异常 Skill 的简化结构如下，`__KB_ID__` 是课程安装流程替换的占位符，手工创建时应填当前 Workspace 的实际 KB ID：

```
---
name: order_exception_recovery
description: 诊断订单缺货与超时问题，准备可审批的履约恢复方案
allowed-tools:
  - search_knowledge_base
  - get_order
  - get_inventory
  - get_shipping_options
  - create_inventory_transfer
allowed-kbs:
  - __KB_ID__
disable-model-invocation: false
requires_network: true
---

1. 检索恢复 SOP，并查询订单、库存及物流事实。
2. 核验状态、可用量与时间约束，列出证据及缺失信息。
```

```
3．满足条件时准备调拨参数与稳定幂等键，进入人工审批。
4．根据实际工具结果报告待审批、执行成功或失败。
```

追问准备：

- **“Skill 和 Tool 有什么关系？”** Tool 提供可调用的能力与参数契约，Skill 描述如何组合能力完成任务；业务函数、HTTP 接口和审批机制仍在执行层。
- **“显式指定 Skill 会绕过权限吗？”** 显式指定影响触发选择，发布绑定、网络授权及调用时的 Tool/KB 校验仍生效。
- **“disable-model-invocation 怎么用？”** 设置后默认不向模型开放自主选择；课程显式选择入口会将指定 Skill 加入当前可用范围，随后通过 skill 工具加载。
- **“为什么加载后还检查权限？”** 工具描述的可见范围和真正的执行授权需要同时落实，防止生成不可用工具名或越权 KB 参数。

代码与资料：Skill 解析器、发布与绑定、Backend 与显式选择、Middleware 与调用校验、完整订单异常 Skill。

Q85｜沙盒有哪些隔离方法？你的项目具体怎么做？

来源：**学员实面题**，U01 第 5 题。

参考回答：

我会从隔离强度和具体边界两方面回答。方法上有受限进程、容器、gVisor 和虚拟机或 microVM；独立进程分离地址空间，容器借助 namespaces 和 cgroups 隔离视图并限制资源，gVisor 增加用户态应用内核，虚拟化方案提供独立来宾内核。选择时还要考虑兼容性、性能和运维成本。

项目把 Python/Bash 执行放到独立 Sandbox Runner，每次创建一次性 Docker 容器。Runner 内部接口鉴权并限制请求，资源配置由服务端决定。容器使用非 root、只读根文件系统、默认禁网、cap-drop ALL 和 no-new-privileges，临时写入限制在 tmpfs，代码由 stdin 传入；CPU、内存、PID、时长、代码量、输出和并发都有上限。

超时或取消后，项目会显式清理容器，正常执行自动删除，启动还会清理孤儿任务。我会验证禁网、写根目录失败、非 root、资源耗尽及取消清理，并区分参数测试和实际容器测试。Docker 与宿主共享内核，Runner 的 Docker 控制权限要单独管理；当前没有接 gVisor 或 microVM，也没有自定义 seccomp 配置。我不会把容器限制描述成绝对安全边界。

不同层次的隔离：

方法	原理与适用考虑	课程情况
独立进程	分离地址空间，仍需文件、用户、系统调用与资源限制	Runner 与 App/Worker 分离，代码进一步进入容器
容器	namespaces 隔离进程、挂载与网络等视图；cgroups 限制资源，配合权限及系统调用策略	已实现一次性 Docker 容器与限制参数
gVisor	用用户态应用内核处理大量系统调用，减少直接暴露给宿主内核的接口	可评估的扩展，需要验证兼容性和性能
虚拟机 / microVM	通过虚拟化边界提供独立来宾内核，管理镜像、启动与资源成本	可评估 Firecracker 等方案，当前未接入

课程实现与验证点：

边界	实际措施	应验证的失败行为
Runner 入口	内部 Bearer Token、请求体上限、语言白名单；资源配置由 Runner 控制	无凭据、额外参数、超大代码及未知语言被拒绝
用户与权限	UID/GID 65534、`--cap-drop ALL`、`no-new-privileges`	无法获得额外权限或访问宿主敏感资源
文件系统	`--read-only`，受限 `/tmp` tmpfs，代码通过 stdin 注入	写根目录失败，临时存储有上限，任务间无共享工作目录
网络	默认 `--network none`	无法直接访问外部业务、云元数据或其他任务网络
资源	默认 256 MiB 内存、0.5 CPU、64 PID、30 秒、64 KiB 代码、stdout/stderr 各保留 1 MiB	内存膨胀、进程爆炸、死循环及大量输出被约束
容量与清理	默认并发上限 4，满载快速失败；正常自动删除，取消/超时显式删除，启动清理孤儿容器	不无限堆积任务，超时后容器及资源被回收

上述数值是最终版 `DefaultConfig` 的默认值，实际环境可以调整。Runner 控制面访问 Docker daemon 所需权限应单独管理，不能将 Docker socket、宿主工作目录或业务根凭据挂给代码执行容器。系统调用过滤采用何种策略应核对实际 Docker 配置；课程没有编写专用 seccomp 配置文件。`/tmp` 的 noexec 仅限制直接执行挂载文件，解释器仍能读取代码，因此其他隔离措施必须共同生效。

追问准备：

- **“只用 Python 虚拟环境或超时够吗？”** 虚拟环境主要隔离依赖，超时限制执行时长；不可信代码还需要操作系统权限、文件、网络及资源边界。
- **“杀掉 docker 命令是否等于容器停止？”** 需要检查容器状态并显式清理，课程在取消与超时分支执行删除。
- **“怎样证明隔离有效？”** 区分参数单元测试与真实容器测试，验证禁网、只读、非 root、资源耗尽、取消清理和跨任务数据隔离；未运行的验证如实标明。

代码与资料：Sandbox 实现、Runner HTTP 契约、隔离测试、Docker 安全 T14、容器运行参数 T15、gVisor T17、Firecracker T18。

Q86 | 说一下 A2A 协议，以及你在项目中是怎样接入的?

来源：**学员实面题**，U01 第 6 题。

参考回答：

A2A 用于不同 Agent 系统之间发现能力、交换消息和协作执行任务，通信双方可以保留各自的内部实现。我会先介绍 AgentCard、Message/Part、Task 和 Artifact：Card 描述接口与能力，Message 承载交互内容，Task 表达需要跟踪状态的任务，Artifact 保存输出产物。协议绑定决定这些操作怎样落到 JSON-RPC、HTTP/JSON 或 gRPC。

课程第 21 课用外部物流 Mock 演示，最终版把远端 Agent 注册成 source_type=a2a 的 Tool。执行器读取 Card，按 supportedInterfaces 顺序选择 JSONRPC，检查接口与 Card 同源并落实网络策略；再归一化 Message，补 messageId，按接口声明加入 tenant 与 A2A-Version，调用固定版本的 SendMessage。响应经过错误与大小检查后回到本地循环。

当前实现范围固定在 v1.0.1 的相关调用，完整任务查询、订阅、推送和签名验证仍需扩展。接长任务时我会继续判断响应类型和任务状态，不能把收到响应当成完成。MCP 负责工具等能力接入，A2A 负责 Agent 交互，Multi-Agent 描述协作架构；远端建议也不能自行获得本地退款或赔付权限，后续动作继续走本地门禁。

需要掌握的知识点：

概念	面试表达
AgentCard	描述 Agent 的能力、接口、协议版本及相关安全信息，供客户端发现和选择；声明仍需验证
Message / Part	一次消息及其内容单元，可承载文本等不同类型内容；`messageId` 标识消息
Task / contextId	Task 表示可持续跟踪的任务，contextId 用于关联交互上下文；两者与本地 Conversation ID 的映射需要应用维护
Artifact	Agent 产出的结果对象；收到响应后仍需验证结构、来源及业务含义
协议绑定	将操作映射到 JSON-RPC、HTTP/JSON 或 gRPC 等通信方式；具体字段和方法按协商版本核对
流式与异步	状态及产物可以通过消息流、任务查询或通知交付，客户端按对端能力选择；课程当前只接 SendMessage
身份与授权	校验服务身份、鉴权凭据、调用范围与数据出口；远端 AgentCard 或回复不能自行扩大本地权限

课程接入链路：

1. 在 Tool 配置中写入 `card_url`，完成版本、鉴权及网络配置，并绑定到允许出网的 Agent。
2. 执行器读取 AgentCard，按 `supportedInterfaces` 顺序选择第一个可用 JSONRPC 接口；检查 Card 与接口同源，并继续执行统一网络策略。
3. 将简单文本或完整 Message 归一化为请求，补齐缺失 `messageId`；按接口声明写入 tenant 和 `A2A-Version`。
4. 使用固定版本对应的 `SendMessage` 方法请求远端，处理 HTTP/JSON-RPC 错误及响应大小限制，将结果交回本地工具循环。
5. 将远端结果作为外部信息核验；涉及本地库存、退款或赔付的动作，仍由本地业务工具与审批决定。

追问准备：

- **“与 MCP、Multi-Agent 有什么关系？”** MCP 标准化工具等能力接入，A2A 标准化 Agent 间交互，Multi-Agent 描述协作架构；团队可在同进程实现，也可通过 A2A 对接远端成员。
- **“长任务只用 SendMessage 够吗？”** 需依据响应类型和任务状态设计后续查询、订阅、取消与结果保存；当前适配器返回调用结果，尚未实现完整远端任务生命周期管理。

- **“AgentCard 的 skills 就是 SKILL.md 吗？”** AgentCard 的 AgentSkill 用于对外描述能力；kbot SKILL.md 用于内部任务说明、加载和授权，两种对象需要分别理解。
- **“不同资料里的方法名不一致怎么办？”** 明确协议版本及 binding，以固定依赖、对端 Card 和契约测试为准；本课程当前采用 PascalCase `SendMessage`。

代码与资料：A2A 执行器、A2A 契约测试、最终版集成范围、第 21 课物流示例、A2A 官方规范 T08。

Q87｜你怎样把 Agent 嵌入到现有项目中?

来源：**学员实面题**，U01 第 7 题。

参考回答：

我会从业务入口、运行服务和业务能力三段接入。入口上，在订单或案件页面提供助手，把用户请求和受控资源标识交给后端；后端验证身份、Workspace、资源与会话归属，再调用固定版本的 Agent。企业已有 SSO 或权限模型时，需要明确身份映射，不能仅依靠前端传入一个 Workspace ID。

能力侧，我把现有查询、规则计算和写操作封装成 Tool，给出 Schema、鉴权、超时和业务前置条件；知识库提供规范，Skill 组织任务。项目已经用独立 HTTP 业务服务接入电商和保险模拟器。Runtime 通过 REST 或 SSE 暴露调用，保存会话，模型根据工具与 KB 证据生成答案；敏感写入产生审批卡片和 checkpoint，批准后 Worker 恢复执行。

以前端订单页为例，助手查询订单、库存和物流，输出有依据的方案；调拨批准后，页面以真实工具结果和业务状态更新。保险案件同样查询保单、责任和风险，赔付、冻结、立案分别受控。SSE 客户端还要区分完成、错误和等待审批，复用 Conversation ID；课程基础 Go SDK 的完整 data 事件需要扩展或按原始契约读取。

当前已跑通管理端和模拟服务，企业身份映射、真实接口、库存事务和持久化业务幂等需要按落地范围补齐。我会用页面发起、工具参数、审批前零写入、批准后状态变化及一次失败分支验收接入，确保它形成可核对的业务闭环。

按七层讲清实现：

层次	具体做法	本课程对应
场景入口	先确定触发点、用户目标、输入与最终状态，如“诊断订单”或“审核案件”	Admin 会话入口、场景 Agent 与任务模板
身份与权限	将业务身份映射到 Workspace/角色，校验资源归属和会话所有者，凭据由受控后端管理	JWT、Workspace 中间件和 Runtime 归属校验；企业 SSO/BFF 映射需扩展
调用接口	短任务同步，交互任务用 SSE，长任务按需要异步化并查询状态	REST、SSE、Go SDK；已有异步 Worker 支撑审批等任务
运行配置	配置 Prompt、模型、Tool、Skill、KB，发布并固定版本，定义上下文与取消策略	AgentVersion、Conversation 与 Runtime Engine
业务能力	以明确契约调用原业务服务，检查参数、状态、金额、超时与重试	两个独立 Go Module，通过 HTTP Tool 接入
副作用控制	高风险写入经过审批，执行时再校验状态并复用稳定幂等键	checkpoint、A2UI、审批 Worker；真实业务持久化幂等需补齐
结果与验证	依据工具和业务状态更新页面，关联 Trace、审计与 Eval，支持排障	控制台事件流、Langfuse 深链、审计及两套 E2E

课程已有接口： 同步入口为 `POST /api/v1/agents/{agent_id}/chat`，流式入口为 `POST /stream/agents/{agent_id}/chat`。最终版请求带 `Authorization` 和 `X-Workspace-ID`，流式请求可传 `message`、`conversation_id`、`agent_env` 及已绑定的用户模板变量。后端继续验证用户及资源权限，Header 中的 Workspace 标识本身不构成授权。

前端处理 `started`、`tool_call`、`tool_result`、`skill_trigger`、`a2ui`、`answer_delta`、`error` 与 `done` 等事件，区分正常完成和等待审批；再次提问复用本会话标识。课程 Go SDK 提供基础同步及流式调用，但当前 Stream 事件类型只暴露 `type/text`，完整会话标识、审批卡片等 `data` 字段需要扩展 SDK 或按原始 SSE 契约读取。浏览器原生 EventSource 不支持这里所需的任意 POST body 与自定义 Header，接入时可使用 fetch 流或经认证的后端代理。

结合两套业务说明：

- **海外电商：** 订单页提供订单号，Agent 查询订单、库存和物流，输出证据与恢复方案；调拨或退款进入审批，执行后以业务服务返回状态更新页面。历史系统接入时补齐真实 OMS/WMS 契约与库存事务。
- **保险：** 案件页提供案件号，Agent 查询案件、保单、责任计算和风险信号，说明可赔上限及待核验项；赔付、冻结、立案分别审批。真实接入时补齐人员授权、条款适用、材料权限与支付幂等。

追问准备：

- **“把 Agent 服务部署起来是否就算嵌入完成？”** 验收必须覆盖入口、身份、工具、实际业务结果、异常分支与审计；部署只是其中一环。
- **“能让模型直接连生产库写 SQL 吗？”** 优先暴露最小业务接口，复用现有状态机与事务。确有 SQL 场景时另设只读账户、表范围、查询预算和专项审核。

- **“用户断线或重复提交怎么办？”** 区分交互请求取消与持久化任务，保留状态；重复提交按业务意图去重，写操作依赖持久化幂等及结果查询。
- **“怎么证明你真的做过？”** 演示从页面发起任务，到工具参数、审批前零写入、批准后业务状态变化，再到 Trace/审计；同时展示一次工具失败或权限拒绝。

代码与资料：API 路由、SSE 请求与事件写入、Go SDK、Runtime 与事件类型、业务工具资产、跨境 E2E、保险 E2E。

1.12 登录检索增补：Agent 岗位与相关工程真题（Q88–Q116）

2026-10-13 使用已登录的牛客网页面核对 10 篇新增个人面经正文，按考点去重后补充以下 29 道题。S11–S18 包括明确的 Agent、AI 应用及相关岗位；S19 为全栈实习，S20 为社招中的 Agent 项目追问，分别保留岗位范围。原帖中标注“推断”、题面缺失或名称无法确认的项目未作为新题依据。S19 作者说明经过 AI 整理，相应问题保留这一证据限制。

复习顺序： Agent 应用岗位先练 Q97–Q110、Q112–Q115；偏后端岗位增加 Q88–Q96；现场编码增加 Q95、Q111，并用 Q116 练习说明验证方法。新增题与既有题互相引用，保留原有 Q01–Q87 编号。

Q88｜做 Agent 应用时，开发流程和传统后端有哪些差异?

来源：**公开面经题**，火山引擎秋招一面 S11，“项目”中的开发方式追问。

参考回答：

我会先从输入和执行路径说明差异。传统业务接口通常有明确的参数和状态转换；Agent 接收自然语言，要判断意图、补充信息、选择工具，路径和调用次数存在变化。因此我在原有接口测试之外，还要评测意图理解、证据使用、工具选择以及失败后能否正确停止。同时，数据库事务、权限和幂等这些后端要求仍然需要落实。

我的开发顺序是先定义业务成功条件，再确定工具契约和允许的操作范围，然后接模型。以保险案件为例，责任计算由业务服务输出确定性结果，Agent 负责组织查询与解释；赔付需要审批，执行时再次检查案件状态。我会先用模拟数据验证工具，再用固定案例检查完整运行轨迹，避免只凭一条自然语言答案判断功能完成。

工程上，我增加了 Prompt、模型、Tool、Skill 的版本绑定，以及 Token 成本、最大迭代次数、工具错误和评测门禁。版本变化后使用同一批案例回归，出现退化能够定位和回滚。我的项目已经有这些平台能力和两套业务模拟器；真实企业接入还需要补齐业务数据权限、线上评测样本与容量验证，不能把本地演示结果当作生产收益。

追问准备： 用一次“订单查询成功，但调拨工具被拒绝”的轨迹说明：接口可用性、Agent 判断质量与业务授权分别怎样验收。关联 Q05、Q59、Q76。

代码/练习入口：Runtime 装配、业务资产。

Q89｜限制使用 MQ 等中间件，怎样设计可恢复的异步任务系统?

来源：**公开面经题**，火山引擎秋招一面 S11，异步系统场景设计。

参考回答：

我先确认能否使用数据库，以及任务量、延迟和可靠性要求。如果允许 PostgreSQL，我会把任务表作为持久化队列：请求通过鉴权和参数检查后，在事务中写入业务意图与任务，提交成功才返回任务 ID 和受理状态；客户端通过状态接口查询。只启动一个 goroutine 后立即返回，会留下进程退出后任务丢失的问题。

Worker 按可执行时间领取任务，通过短事务、行锁及 `SKIP LOCKED` 避免重复领取，记录租约和执行次数，再在事务外执行耗时操作。成功后写结果；可重试错误按退避时间重新排队，参数错误等永久失败直接结束。超时任务由扫描器检查租约后重新调度。任务已经执行但结果未写回时仍可能再次执行，所以业务写入必须使用持久化幂等键，并能查询原结果。

放到 kbot 的知识库入库场景，我会保存文档版本、当前阶段和错误原因，支持查询与重试；审批恢复则需要保存获批的具体操作及恢复地址。课程当前使用 Redis/Asynq 承载异步任务，并有入库阶段记录；数据库任务队列属于这个限制条件下的替代设计。我会用"领取后崩溃""执行后崩溃""租约过期与旧 Worker 同时返回"等故障验证状态及副作用，必要时增加租约版本防止过期 Worker 覆盖新结果。

追问准备： 如果连持久化存储也不允许，说明单机内存队列能提供的范围，以及重启后恢复任务所需的日志与重放机制。关联 Q49、Q51。

代码/练习入口：现有异步任务、入库阶段记录。

Q90｜调用模型或工具收到 HTTP 429，如何判断和处理?

来源：**公开面经题**，火山引擎秋招一面 S11，HTTP 状态码追问；处理方案结合课程扩展。协议依据 T25。

参考回答：

我先确认 429 来自哪一层：应用入口、模型供应商、业务工具还是沙盒容量限制，再看响应体、请求 ID 和限额维度。429 表示一定时间内请求过多，服务端可能提供 `Retry-After`；具体是请求频率、Token 配额还是并发容量，需要结合对方契约判断。鉴权失败、参数错误和业务审批拒绝有各自的处理方式，不能统一进入重试循环。

对允许重试的调用，我会优先尊重等待提示，缺少提示时用带随机抖动的指数退避，同时限制总次数和总耗时，并响应 context 取消。多个请求同时遇到限额时，需要入口背压和共享预算，避免每个请求独立重试放大压力。查询通常可以重试；写操作先核对是否已产生副作用，再通过业务幂等或结果查询决定后续动作。

课程有进程内令牌桶和 Redis 固定窗口等限流构件，沙盒容量错误会映射为 429，但各个 Guard 拒绝都返回 429 的统一契约尚未完成。我会补齐错误分类、限额维度和可重试标记，观察拒绝率、排队时间及恢复后的成功率。另外，当前 Redis 计数与设置过期分成两步；生产强化时应使用原子脚本处理，避免中途失败留下没有过期时间的计数键。

追问准备： `Retry-After` 可以采用秒数或 HTTP 日期（T26）；客户端时钟偏差、总超时和已取消请求都要纳入等待判断。

代码/练习入口：限流构件、沙盒 HTTP 错误映射。

Q91｜进程、线程和 Go 协程怎样区分，Agent 并发应该怎样控制?

来源：**公开面经题**，火山引擎秋招一面 S11，进程/线程及 Go 协程追问。概念校核见 T21。

参考回答：

我会按资源边界和调度方式区分。进程拥有独立的虚拟地址空间等资源，线程共享进程内的地址空间，由操作系统调度；goroutine 是 Go Runtime 管理的执行单元，多个 goroutine 可以复用操作系统线程。goroutine 初始栈较小且可增长，适合组织大量并发工作，但实际开销仍受栈、对象引用、网络连接和调度影响。

在 Agent 场景里，模型请求和业务 HTTP 调用往往等待外部 I/O，使用 goroutine 比较自然。需要同时查询几个互不依赖的仓库时，我会给任务设置并发上限、超时和取消，收集每个查询的独立结果；后一步依赖前一步状态时则顺序执行。CPU 密集的本地计算还需要看核心数和 `GOMAXPROCS`，单纯增加 goroutine 数量无法保证吞吐提升。

项目已经通过 context、Worker 并发配置和部分执行容量控制约束资源，Supervisor 当前按顺序调用成员。我扩展并行工具时，会先检查服务端限额与共享状态，再用有界任务组控制数量，确保返回路径关闭资源、错误路径不会泄漏 goroutine。压测会同时看吞吐、P95、连接池等待、内存和 goroutine 数，不能只展示“同时起了多少协程”。并行写操作还需要业务事务和幂等保障。

追问准备： 并发描述多个任务的推进关系，并行还涉及实际同时执行；用等待模型响应和本地批量计算两个例子解释区别。

代码/练习入口：Worker 并发配置、Team Runtime。

Q92 | Redis 怎样持久化？用了 AOF，还需要关系数据库吗？

来源：**公开面经题**，火山引擎秋招一面 S11、恒生电子 AI 应用一面 S14，Redis 持久化及数据丢失追问。校核见 T22。

参考回答：

我先区分 RDB 和 AOF。RDB 保存时间点快照，文件紧凑、恢复方便，但故障时可能丢失上次快照后的修改。AOF 记录写操作，可靠性与刷盘策略有关：`always`、`everysec` 和由操作系统决定刷盘，各有性能与故障窗口取舍。`everysec` 通常可能损失约一秒写入，具体还受到磁盘和故障条件影响，我不会承诺绝对零丢失。

持久化解决重启恢复的一部分问题，数据模型、事务约束和查询能力仍要单独选择。我的项目把 Workspace、资产版本、会话、审批等关系数据放 PostgreSQL，Redis 用于异步队列和运行治理。涉及赔付审批时，要查询案件、审批状态、授权和操作记录，关系约束及事务边界更容易表达；Redis 持久化开启后，这些业务建模工作仍然存在。

我还会区分备份、复制和持久化。主从复制存在传播延迟，故障切换时的可接受损失要实际评估；备份要验证能否恢复。课程使用 Redis/Asynq，不代表业务模拟器的幂等 map 已经持久化。生产接入时，我会把业务意图及幂等结果放入可靠存储，检查 Redis 配置、磁盘与恢复流程，并演练队列重投后业务操作能否安全重复。

追问准备： 从“应用已收到成功响应，随后机器断电”的时间线，解释刷盘、复制、业务提交与客户端确认之间可能出现的窗口。

代码/练习入口：部署配置、异步任务。

Q93 | 异步任务大量积压，怎样监测、定位和处理？

来源：**公开面经题**，火山引擎秋招一面 S11，消息积压及观测追问。

参考回答：

我会同时看积压数量、最老任务等待时间、进入速率、完成速率和重试率。数量少但等待很久也可能违反时效要求，因此我会按任务类型和优先级分别统计。判断原因时，先区分流量突然增加、Worker 不足、单任务变慢、下游限流，以及坏任务反复失败，再结合处理时长与错误分布定位。

如果下游容量充足，我可以增加 Worker 或缩短单任务的无效等待；遇到模型限流、数据库连接池满或业务接口变慢时，要先收敛并发并处理瓶颈。永久错误应隔离，重试需要退避和上限。对昂贵入库任务可以按文档版本合并重复请求，对审批恢复则保护时效，避免被批量任务占满。扩容必须同时检查公平性、重复执行与数据库压力。

课程的 Asynq 配置已有 critical、default、low 队列权重和任务超时、重试，但完整积压监控与告警策略还需补充。我会先建立基线，用压测或故障注入观察模型变慢后队列如何增长，再制定告警与恢复阈值。清理积压前要区分过期任务、可重放任务和产生过副作用的任务，保留处置记录；对电商调拨和保险赔付，不能通过简单重放制造重复操作。

追问准备： 用“到达速率持续高于完成速率”的例子解释积压为何增长，并说明扩容为何可能加重下游限流。

代码/练习入口：队列、超时与重试、入库阶段与 Trace。

Q94｜从客户端 curl 到网关和 Agent 服务，一次请求如何往返?

来源：**公开面经题**，火山引擎秋招二面 S12，客户端、服务端与网关场景题。

参考回答：

我先确认协议、代理和连接是否复用。以新建 HTTPS、HTTP/1.1 连接为例，客户端解析域名，根据路由决定下一跳；同网段通信或访问网关时，通过邻居解析获得下一跳链路地址。随后完成 TCP 建连和 TLS 握手，发出 HTTP 请求。DNS 缓存、连接复用、HTTP/2 多路复用或 HTTP/3 都会改变部分细节，所以我会明确回答前提。

请求到反向代理后，通常会终止一段连接，再建立或复用到后端的另一段连接；应用层网关和网络默认网关承担不同职责。kbot 应用收到请求后验证用户、Workspace 和资源权限，装配 Runtime，再调用模型、知识库或业务工具，返回普通 JSON 或 SSE。SSE 是同一响应持续发送事件，代理缓冲和空闲超时可能影响用户看到首个事件的时间。

排障时，我会沿客户端、代理、应用、模型和工具记录同一请求标识，分别检查解析、建连、握手、首字节和业务执行耗时。客户端断线后还要确认 context 取消能否传到下游，以及已经入队的持久化任务如何继续。课程里可以从 API 路由、SSE 和 Trace 验证应用链路；实际网络拓扑及 TLS 终止位置需要结合部署说明，不能凭“用了网关”推断全部行为。

追问准备： 返回路径按当前各段连接和网络路由处理；从浏览器到应用、从应用到模型供应商是两条独立的请求链。

代码/练习入口：API 路由、SSE。

Q95｜实现 LRU 缓存时，如何保证查询和更新的时间复杂度?

来源：**公开面经题**，火山引擎秋招一面 S11、抖音电商三轮面经 S13，均记录了 LRU 编码题。

参考回答：

我会用哈希表加双向链表。哈希表把 key 映射到链表节点，链表维护最近访问顺序：头部放最近使用项，尾部放最久未使用项。Get 命中后把节点移到头部；Put 更新已有节点时同样移动，新增后超过容量就移除尾节点，并删除哈希表记录。这几个操作的平均时间复杂度都是 O(1)，空间复杂度为 O(capacity)。

实现时我倾向使用哨兵节点，统一处理空链表和单节点情况。我会先确认容量为零、未命中返回值及重复 Put 的语义，再测试“访问后改变淘汰顺序”“更新值但容量不变”“连续淘汰”。Go 可以使用 `container/list`，也可以手写节点；面试要求手写时，我会说明断开、插入节点的指针更新顺序，避免形成环或留下过期映射。

如果用于 Agent 平台缓存，缓存键还要包含 Workspace、数据版本和模型版本等必要维度，先保证不会跨租户复用结果。LRU 只控制容量淘汰，过期时间和数据失效需要额外设计；并发访问需要同步，耗时的模型或数据库请求应放到锁外，并考虑合并相同未命中请求。课程没有把这道算法实现列为已交付的业务能力，我会用独立练习及边界测试展示掌握程度。

追问准备： 解释 LRU 与 TTL 的区别，以及大批一次性访问可能怎样挤掉热点数据。关联 Q77–Q79 的现场编码训练。

Q96｜一亿条业务记录需要分批调用 RPC 更新，怎样保证能持续推进并恢复?

来源：**公开面经题**，抖音电商三轮面经 S13，一面大批量子订单处理场景。

参考回答：

我先确认筛选条件、是否存在并发修改、RPC 是否支持批量和幂等，以及允许的完成时限。扫描层使用稳定唯一键做游标分页，并固定本轮数据范围，例如记录开始时的最大 ID，避免大量 OFFSET 和新增记录让扫描范围不断变化。对筛选条件会变化的数据，还需要快照、版本或独立任务清单，防止扫描期间漏掉目标。

执行层按批读取、限制 RPC 并发和请求速率，保存批次状态、成功项、失败项及重试时间。游标推进前，要确保本批结果和未完成任务已经可靠记录；进程重启后从这些记录继续。RPC 超时表示结果未知，我会先通过幂等键或结果查询确认是否已执行；批量接口部分成功时，按明细重试失败项。业务更新还需要条件检查，避免用旧状态覆盖用户的新操作。

映射到课程，可以扩展为海外电商历史订单对账或保险存量案件核验，复用 Worker、任务状态和审计能力，但当前项目没有一亿条数据的性能验证。我会先用可控样本测每批耗时及下游承载，再估算完成时间，并演练中途重启、部分失败和重复执行。验收时核对目标总量、成功、失败、跳过与待处理数量，抽查业务结果，保留重跑入口。

追问准备： 游标分页需要稳定排序；如果按更新时间排序，应增加唯一键作并列排序，并处理扫描中更新时间变化。关联 Q89、Q93。

代码/练习入口：异步任务基础、跨境业务服务。

Q97｜材料含糊、字段名称相似，怎样区分已确认事实和待核实关系？

来源：**公开面经题**，抖音电商三轮面经 S13，二面合同审核中的关系识别与不确定输入。

参考回答：

我会把抽取结果分成已确认、存在冲突和证据不足三类，分别保存原文位置、候选解释和缺失条件。名称相似只说明可能相关，关系成立还要看主体、时间、适用范围和明确引用。模型给出的自评分数可以作为观察信息，只有经过标注数据校准后，才能用来制定可靠的自动处理阈值。

例如保险材料里同时出现相似的保单号或附加条款，我会先从结构化接口取得案件绑定的保单，再核对事故时间与条款版本。能通过规则确定的字段做精确比对；存在歧义时列出候选及证据，询问用户或进入人工复核。金额、主体和责任条件缺失时，要保留未知状态，禁止随意补默认值后继续执行赔付。解释层可以归纳材料，决策层仍检查业务前提。

课程已有案件查询、条款核验、责任计算和审批工具，但完整的合同关系抽取及置信度校准属于扩展。我会建立相似名称、错误编号、缺日期和互相矛盾材料的样本，分别衡量正确确认、误确认与澄清比例。每个结论要能追溯到原始证据，人工修正进入后续标注集；这样面试时可以清楚说明“何时自动处理、何时必须停下来补信息”。

追问准备： 模型自信与证据充分分别怎样衡量？错误确认造成业务写入时，哪些服务端检查还能拦住？关联 Q29、Q46、Q73。

代码/练习入口：保险业务服务、保险业务说明。

Q98｜几百页合同超过上下文窗口，怎样拆分处理并避免遗漏跨章节条件？

来源：**公开面经题**，抖音电商三轮面经 S13，二面长合同、分块及层次聚合追问。

参考回答：

我会先区分任务：查一条规定可以检索相关段落，要求完整审查合同则需要遍历与覆盖检查。先解析目录、章节、页码、定义和引用关系，再按实际模型的 Token 预算拆分，为系统指令、证据、工具结果和输出预留空间。页数无法直接代表 Token 数，固定按页切也容易把表格和同一条款拆开。

分层处理时，子任务提取结构化事实、风险、适用条件和原文坐标，父任务按统一 Schema 合并，并显式保留冲突和缺失项。定义、免责条款及附件建立引用索引，遇到跨章节依赖就回查原文。仅汇总子任务摘要可能丢失例外条件，因此我会让全局检查遍历必须覆盖的条款清单，记录每一项由哪些证据支持，无法核验的结论进入待复核状态。

放到保险项目，我会先做保单主体及定义抽取，再按责任、免责、限额等维度审核，最后由确定性工具计算金额。课程当前只有基础切片和检索，通用长合同解析、分层执行及覆盖报告尚需实现。我会用跨页表格、后文否定前文、附件变更限额等样本验证完整性，并测调用次数、成本和漏检率，确保增加层次处理确实改善结果。

追问准备： Map-Reduce 的每层都需要预算与可追溯证据；扩展为子 Agent 时还要控制并发、权限、重试和中间产物版本。关联 Q38、Q112。

代码/练习入口：当前切片逻辑、Team Runtime。

Q99｜负责评判的 Judge Agent 也会产生幻觉，怎么校验它的结论?

来源：**公开面经题**，抖音电商三轮面经 S13，二面事实依据及评判模型可靠性追问。

参考回答：

我会把校验分成结构、事实、规则和主观质量四层。结构层检查输出 Schema 和必填字段；事实层检查引用位置是否存在、订单或案件状态是否与工具结果一致；规则层用程序核对金额、时间及允许的操作；解释是否清晰、方案是否合理这类主观质量再由模型辅助评价。Judge 输出也要带依据和无法判断的原因。

例如保险 Agent 声称“赔付计算正确”，我会用业务计算器独立重算限额、免赔额和比例，并检查所引用条款的版本。Judge 对语言表达的评价无法替代这些校验。对于难以程序化的结论，我会用人工标注样本测 Judge 与人工的一致性，检查漏判和误判；必要时进行盲评、交换答案顺序或多次复核，观察位置偏好和不稳定性。

课程已有确定性 Judge 和 LLM Judge，我会优先让确定性规则承担硬约束，让 LLM Judge 提供解释或软评分，再由门禁决定是否发布。当前业务样本规模很小，还不足以证明 Judge 在真实场景中可靠。若扩展合同审查，我会单独建立 Judge 校准集，保存评判模型、Prompt、证据和人工复核结果，避免被评模型与评判模型共同遗漏同一条限制。

追问准备： 正确答案、完整执行轨迹和 Judge 自评分数分别能证明什么？没有证据时，评判器应如何表达无法判断？关联 Q46、Q59、Q60。

代码/练习入口：Judge 实现、评测服务。

Q100｜CLI、MCP 和 Skill 怎样选？接入方式怎样影响调用性能?

来源：**公开面经题**，抖音电商三轮面经 S13，二面 CLI/MCP 对比及约 51:51 的多入口耗时讨论；原帖对“何时需要 Skill”的推断条目未作为独立真题证据。

参考回答：

我先按职责拆开：CLI 是程序的命令行入口，MCP 统一能力发现和调用交互，Skill 提供可按需加载的任务说明、流程及相关资源。它们可以组合，例如 Skill 指导 Agent 调用某个 MCP 工具，工具内部再执行受控 CLI。选择时，我会看部署位置、客户端数量、发现需求、参数稳定性和权限边界。

已有稳定的本地命令行工具时，可以用执行器包装参数、超时和结构化输出；需要多个 Agent 客户端接入时，MCP 更方便统一契约；需要复用业务操作流程时，用 Skill 描述步骤和例外条件。CLI 调用要处理进程启动、stdout/stderr 和退出码，MCP 调用要处理连接、协商及远端错误，Skill 本身还可能增加上下文和模型调用。性能比较应拆开冷启动、首次发现、复用连接及单次执行来测。

课程中 Skill 使用渐进披露并收窄 Tool/KB 范围，MCP 主要通过 Streamable HTTP 接入，控制面限制直接配置 stdio。接入本地 CLI 时应放入独立受控 Runner，校验参数并限制文件、网络和资源。我不会直接宣称某一种方式总是更快；会在同一业务动作下测 P50/P95、Token 和调用次数，同时检查出错后是否可定位、可重试和可审计。

追问准备： 工具协议和业务步骤的复用分别解决什么问题？CLI 输出包含不可信指令时，Agent 应怎样处理？关联 Q30、Q32、Q84、Q85。

代码/练习入口：MCP 执行器、Skill Runner。

Q101｜为何要持久化任务与工具执行过程？保存聊天记录够用吗？

来源：**公开面经题**，抖音电商三轮面经 S13，三面任务、工具调用与执行链路设计追问。

参考回答：

我会分别保存对话内容、业务任务状态和执行证据。对话用于还原交互，任务记录当前处于待执行、执行中、等待审批或结束，执行记录解释具体用了哪个版本、传了什么参数、获得什么结果。只看到"退款已申请"这句话，无法确认业务接口是否真正执行，也无法判断重试是否会重复退款。

设计时，我会用 task ID 串联会话、步骤、tool call、审批和业务操作，记录版本、开始结束时间、错误类别及必要的结果摘要。恢复还需要可继续执行的状态和地址；Trace 用于观测，不能直接承担任务队列或业务事务。工具结果很大或涉及敏感材料时，保存受控对象引用、摘要和访问权限，避免所有原始内容无限复制进聊天日志。

课程已有持久化会话、审批 checkpoint、审计及工具执行相关记录，可展示一次审批暂停与恢复。它还没有覆盖任意步骤崩溃后的通用恢复，所以我会明确这个范围。如果升级成长任务系统，要补齐步骤状态、租约和幂等结果核对：执行超时后先查询业务状态，确认副作用，再决定重放或人工处理。这样既能解释发生过什么，也能判断下一步允许做什么。

追问准备： 保存了完整 Trace，为什么仍可能无法安全恢复？关联 Q49、Q50、Q51、Q53。

代码/练习入口：Runtime Engine、审批恢复。

Q102｜相似说法对应不同意图，怎样提高并量化意图识别质量？

来源：**公开面经题**，迅雷 Agent 一面 S15，意图准确率及相近问句区分。

参考回答：

我先把意图与业务动作对应起来，特别区分查询、解释、申请和真正执行。比如"退款到哪了"是查询进度，"这单能退吗"是资格判断，"帮我退这单"才涉及操作申请。识别时同时考虑当前问句、有效的会话上下文和页面业务对象，并检查否定、条件、代词及缺失信息；置信不足或操作对象不明确时先澄清。

量化时，我会建立人工标注集，包含同义表达、近似表达、多意图、无关请求和历史信息变化，用混淆矩阵分析最容易混淆的类别，再分别统计各类 Precision、Recall 和整体表现。写操作误触发的代价更高，需要单独衡量。采用拒识或澄清机制后，还要报告自动覆盖率及人工介入比例，避免通过大量拒答让准确率看起来很好。

课程目前由 Agent 结合上下文和工具描述进行决策，没有经过独立标注集验证的意图分类器。我会先用电商退款和保险赔付做离线试验，比较规则、分类模型及 LLM 结构化路由，并检查后续工具参数和任务结果。历史解决方案只能在对象、状态和版本仍适用时复用，新的事故时间或订单状态应重新查询。高风险动作继续由服务端权限、审批及业务校验兜底。

追问准备： 意图识别正确但参数对象错误时，应该记在哪一项指标？"历史有退款意图"能否授权当前退款？关联 Q07、Q20、Q59。

代码/练习入口：上下文与执行装配、电商业务工具。

Q103｜切片以后怎样保留上下文关系，让检索结果能找到相关章节?

来源：**公开面经题**，迅雷 Agent 一面 S15，Chunk 间关系及索引追问。

参考回答：

我会区分顺序关系、结构关系和显式引用。顺序关系用 document ID 和 ordinal 找相邻片段；结构关系记录章节路径、父节点和所属表格；显式引用记录“见附件二”等链接及目标。这样命中一个片段后，可以有选择地补充前后文、父章节或被引用条款，减少孤立切片造成的误读。

查询时，我先按权限和版本召回候选，再对关键片段做有限扩展、去重和预算裁剪。扩展后的证据继续保留原文位置，不能因为首片段可访问，就默认关联文档也可访问。比如保险责任条款命中后，需要补充定义和对应免责条件；电商 SOP 命中调拨步骤时，需要补充前置库存条件及审批要求。关系扩展要受相关性和 Token 预算约束，避免把整份文档重新塞回上下文。

课程入库已保存文档 ID 和切片顺序，基础切片有 overlap；章节树、引用关系及父子检索属于扩展。我会先增加可追溯元数据，再用跨切片条件、附件引用和版本更新样本验证，比较基础检索与关系扩展后的证据完整率、回答正确性和成本。关系索引更新时，需要同步删除旧版本关联，防止新条款连接到过期例外条件。

追问准备： overlap、相邻扩展、父子检索和知识图谱各自适合保留哪一种关系？关联 Q38、Q39、Q98。

代码/练习入口：切片、索引元数据。

Q104｜检索 Top-K、生成 Top-P 和重排怎样区分，参数怎样调?

来源：**公开面经题**，迅雷 Agent 一面 S15，RAG 中 Top-K/Top-P 及 Rerank 原理追问。

参考回答：

我会先确认面试官说的是检索参数还是生成参数。检索 Top-K 表示返回多少候选片段；生成中的 Top-P 通常指按下一 Token 概率累计选取采样候选，Top-K 也可以用于 Token 采样。同一个名字出现在不同环节，作用对象不同。若某个检索器也定义 Top-P，需要按其具体分数与概率定义解释，不能直接把相似度当作校准概率。

重排处理的是已召回的 query-document 对。Cross-Encoder 将两者联合编码并计算相关性，通常比独立向量打分更细致，也增加计算成本；它无法找回召回阶段遗漏的材料。调参时我会先用标注的相关片段检查 Recall@K，再评估重排后的排序和最终证据，分别设置召回数量、送入重排数量与最终上下文数量，同时观察延迟、噪声和 Token 成本。

课程 PostgreSQL 路径分别取最多 50 条全文与向量候选，再用 RRF 合并并返回指定数量，当前没有独立 Cross-Encoder。RRF 中的平滑常数与最终 Top-K 也应区分。我会把这条实际链路讲清，再提出增加重排的对比实验；原帖里其他候选人的参数只能作讨论材料，不能写成我的实现或最优配置。模型生成的采样参数另外评测，尤其检查金额及工具参数是否稳定。

追问准备： 候选相关但缺少关键免责条款时，增加 Top-K、关系扩展或改进切片分别能解决什么？关联 Q39–Q41、Q103。

代码/练习入口：PostgreSQL 混合检索、重排原理 T13。

Q105｜团队使用不同 Coding Agent，怎样统一开发约定并避免规则漂移?

来源：**公开面经题**，迅雷 Agent 一面 S15，多人开发约定及不同 Coding Agent 入口文件追问。

参考回答：

我会把稳定的工程规范放在版本管理的统一文档里，包括项目结构、运行方式、接口约束、测试要求和敏感操作边界，再为不同工具提供简短的入口说明或受控生成的适配文件。核心规则尽量保持一个维护源，避免每个工具各自复制一份后逐渐不一致。工具读取哪个入口、怎样处理目录范围，需要按实际版本确认。

同时，我会区分团队规范、当前任务和外部材料。团队规则定义工程要求，任务说明确定本次目标，检索内容及第三方仓库只提供证据。模型即使读到了规范，也仍需要代码评审、自动检查和测试验证。我会让成员提交小范围改动，附上运行结果和未解决问题，并在发现规范冲突时明确适用范围，不依赖模型自行猜测团队意图。

放到课程项目，我会统一模块边界、Workspace 鉴权、业务写入审批、幂等和验证方式；海外电商及保险各自保留业务契约。不同 Coding Agent 可以协助改代码，但都要通过相同的接口测试与业务案例。课程没有交付跨工具规则同步器，若继续建设，我会增加入口一致性检查，并用几项典型任务验证工具是否遵守关键约束，避免把"配置了说明文件"当作执行正确的证据。

追问准备： 自动生成的适配文件怎样检查过期？某条团队规范与本次用户目标冲突时，怎样说明范围并继续完成已授权工作？关联 Q76、Q116。

代码/练习入口：课堂到最终版映射、课程验证要求。

Q106｜知识库支持哪些文档格式？PDF、图片和表格准备怎样处理？

来源：**公开面经题**，经纬恒润 AI 应用一面 S16，上传格式、解析器及图表处理追问。

参考回答：

我先准确说明已实现范围。课程入库的 parse 阶段目前接收 Markdown 和纯文本，内容直接进入后续切片；Markdown Folder Connector 读取 `.md` 文件。这里有入库阶段名称，但还没有原生 PDF、Word 或 OCR 解析能力。我会展示实际输入与代码，避免把流程里的 parse 标签理解成完整文档解析系统。

若扩展 PDF，我会先区分可提取文本的 PDF 与扫描件：前者提取文字及版面信息，后者需要 OCR，并记录页码、坐标和识别质量。表格保留表头、单位、跨页关联及行列关系，不能只把所有单元格串成一段；图片提取图注，必要时交给视觉模型生成带来源的辅助描述。复杂条款和金额材料要保留原件供复核，模型生成的图片描述不能自动升级为已确认事实。

入库前，我会验证文件类型、大小和可解析性，解析后检查乱码、空页、重复页及表格结构，再进入切片、Embedding 和索引。保险材料重点测试盖章扫描件、跨页限额表和模糊金额，电商材料重点测试报价表和多币种数据。评估既看解析字段正确率，也看下游证据召回；当前版本只能提交文本链路的验证结果，新增格式必须完成单独测试后再写进简历。

追问准备： OCR 把金额的小数点识别错了，在哪一层发现、怎样暂停后续赔付？关联 Q37、Q75、Q107。

代码/练习入口：真实 parse 阶段、Markdown Connector。

Q107｜知识库内容怎样审核，避免错误、过期和越权材料进入答案？

来源：**公开面经题**，经纬恒润 AI 应用一面 S16，知识库质量审核及术语确认追问。

参考回答：

我会把质量检查放在入库前、发布时和检索时。入库前检查来源、格式、重复及敏感级别；涉及新术语或业务规则时由负责人确认含义和适用范围。发布时记录负责人、生效时间、版本及审核结果，过期材料需要下线或明确标注。检索时先限制 Workspace 和允许访问的知识库，再检查版本是否适合当前业务对象。

以保险条款为例，同名产品不同年份的责任和限额可能不同，检索到相似条款只能提供候选，仍要核对案件绑定的保单版本。电商报价也需要承运商、线路、币种和有效期。资料互相冲突时，我会保留来源并提示待核验，依靠业务规则或人工复核确定适用内容。答案展示引用位置，有助于用户验证依据，但引用存在本身还不能证明结论成立。

课程有 Workspace、资产版本、KB 权限约束及入库阶段状态，但完整的内容审核发布流、文档生效期过滤和自动失效治理需要扩展。我会先建立一组重复、过期、冲突、越权和错误术语样本，检查是否阻止发布或在查询时排除，并验证更新后旧切片能否清理。线上再通过问题反馈追踪内容责任人，把错误归因到资料、解析、检索或生成环节。

追问准备： “索引成功”和“内容获准发布”分别对应什么状态？如何处理正确资料被错误拆分后造成的答案偏差？关联 Q44、Q46、Q48、Q60。

代码/练习入口：入库状态、混合检索。

Q108｜开放式方案没有唯一标准答案，怎样评估 Agent 做得好不好?

来源：**公开面经题**，小厂 Agent 实习面经 S17，长文档、规划类任务缺少标准答案时的评估。

参考回答：

我会先把业务目标转成可核验的约束，再定义分级评价维度。即使物流方案没有唯一最优答案，库存数量、交付时限、成本上限和可用线路仍可检查；方案的解释完整性、替代选择与异常处理则可以按明确量表评价。这样能分别识别硬约束违规和方案质量差异，避免只让模型给一个笼统分数。

评估时，我会让业务人员对代表性案例做盲评或成对比较，先校准评分标准，再检查评审者一致性。LLM Judge 可以辅助筛选，但需要与人工结果对照。长文档任务还要验证覆盖率和引用支持，规划任务要在模拟或受控环境里检查是否可执行。面对多种合理方案，应允许不同路径通过，只要满足必要条件并说明取舍。

课程两套业务项目可以提供确定性约束和基础 Eval，但当前样本数量较少，尚不足以评价复杂开放任务。我会补充缺货、物流延迟、条款冲突和信息不足等场景，同时保存任务结果、工具轨迹、人工评价、耗时与成本。如果新增方案提高了可读性，却更容易违反库存或金额条件，发布门禁仍应拒绝；质量改进要结合多个维度一起判断。

追问准备： 开放任务的“可接受答案集合”怎样定义？怎样避免评分标准偏向某一套固定措辞？关联 Q59、Q60、Q99。

代码/练习入口：Eval、两套业务资产。

Q109｜Agent 怎样主动发现业务事件并推送，避免反复打扰用户?

来源：**公开面经题**，小厂 Agent 实习面经 S17，主动推送实现追问。

参考回答：

我会把主动推送拆成事件发现、规则筛选、Agent 分析和可靠投递。事件可以来自业务状态变化、Webhook 或定时扫描，例如订单临近超时且仍然缺货。先用规则检查事件是否达到触发条件，再让 Agent 结合订单和 SOP 生成摘要与建议，最后通过用户订阅的渠道发送。触发信息需要带业务对象和版本，便于查询最新状态。

可靠性上，我会记录事件 ID、接收人、推送内容和投递状态，用稳定键去重，按渠道能力处理重试和回执。发送前再次检查事件是否已经解决，设置同类事件冷却窗口、频率上限和用户静默时段。模型生成内容可以失败重试；业务动作仍走正常授权与审批，收到提醒不会自动形成调拨或赔付许可。通知涉及哪些字段，也要按接收人的业务权限筛选。

课程已有飞书入站消息的异步回复及去重链路，可以借鉴任务载荷、稳定 ID 和 Worker 机制；通用的主动事件订阅、冷却与通知中心尚未实现。我会先用海外电商缺货提醒做扩展实验，验证重复事件只投递一次、状态恢复后取消提醒、渠道失败后有界重试，并观察有效提醒比例和用户关闭率。验收既包括消息送达，也包括内容及时性及后续处理效果。

追问准备： 对方渠道不支持幂等且发送结果未知时，应怎样核对与控制重复？关联 Q89、Q93、Q115。

代码/练习入口：飞书异步回复、回复任务去重。

Q110｜怎样比较 LangChain、LangGraph 和 Eino？运行状态应该保存哪些字段？

来源：**公开面经题**，小厂 Agent 实习面经 S17，框架与手写选择；字节 Agent 实习面经 S18，LangGraph 作用及状态字段。框架概念校核见 T03、T24。

参考回答：

我会先根据语言栈、编排复杂度和恢复要求选型。LangChain 提供模型、工具及 Agent 等应用抽象与集成；LangGraph 提供显式状态和图执行能力，适合需要控制节点、分支与长任务过程的场景。课程采用 Go 和 Eino ADK，框架承担模型与工具循环，平台补上版本、权限、审批、计量和业务集成。选型时要验证实际接口与固定版本，框架名称不能代替能力验收。

运行状态我会分为任务身份、对话和证据、执行控制、业务结果。具体包括 task/conversation ID、Workspace 与用户、绑定版本、messages、当前阶段、已完成步骤、tool call 及结果引用、预算、错误与重试次数，以及审批或恢复地址。消息追加与状态覆盖需要明确合并规则，尤其是并行节点返回时；大文件保留受控引用，密钥留在服务端。

课程已有会话 Runtime Snapshot 和审批 checkpoint，但没有使用 LangGraph，也没有实现通用并行 DAG。我会用同一条“查询—发现缺口—审批—恢复”的案例比较候选框架，检查状态持久化、升级兼容、可观测性及恢复后的副作用，而不会只比较 Demo 代码行数。若迁移框架，工具契约、业务状态机及评测样本尽量保持稳定，以便核对行为变化。

追问准备： messages 属于运行状态的一部分；哪些字段必须持久化，哪些可重算，哪些应避免写入状态？关联 Q04、Q12、Q49、Q101。

代码/练习入口：Runtime 装配、Team Runtime。

Q111｜删除重复字符，怎样保持相对顺序并得到字典序最小的结果？

来源：**公开面经题**，字节 Agent 实习面经 S18，算法题；原帖描述概括为常见的去重字符子序列问题。

参考回答：

我会先确认要求：结果是原字符串的子序列，每种字符恰好保留一次，并在这些结果里取字典序最小值。保持相对顺序意味着不能直接排序。例如 `cbacdcbc` 的结果是 `acdb`。如果字符范围包含中文，需要先约定按 Unicode 码点还是业务排序规则比较，再选择合适的遍历和下标方式。

算法用一个栈、已入栈集合，以及每个字符最后出现的位置。扫描当前字符时，已经在栈中就跳过；否则，当栈顶字符比当前字符大，并且栈顶在后面还会出现，就弹出栈顶并更新集合，再把当前字符入栈。能够再次出现保证弹出后仍能补回，字典序比较保证优先保留较小前缀。如果栈顶后面再也不会出现，就必须留下它。

每个字符最多入栈和出栈一次，所以总时间 O(n)，额外空间与不同字符数有关。我会测试空串、全相同字符、无重复字符、严格降序，以及“栈顶更大但后面不再出现”的情况。这道题主要证明我能解释贪心选择和数据结构边界；课程项目里业务标识的去重通常保留语义及原始顺序，未经业务要求确认，不能直接套用这个字典序规则。

追问准备： 现场用 `bcabc` 推演弹栈过程，再说明如果删掉“后面仍会出现”条件会丢失什么字符。

Q112｜子 Agent 的消息和工具权限如何隔离，怎样避免继承过多能力？

来源：**公开面经题**，美团全栈实习面经 S19，子 Agent 工具与消息范围；作者说明题目经过 AI 整理。

参考回答：

我会分别设计输入范围、执行身份和可用能力。父 Agent 给子 Agent 一个明确任务，只传相关对象、必要证据和约束；子 Agent 使用自身绑定的模型、Prompt、Tool 和 KB 范围。用户与 Workspace 身份贯穿调用，具体业务接口继续校验授权。父 Agent 有某个高风险工具，也不能据此推导所有子 Agent 都能使用它。

课程 Supervisor 把固定成员包装为 AgentTool。成员执行入口提取最近一条非空消息，再通过自身 Agent ID 和版本进入 Engine.Chat，并带上 Workspace 与用户；成员工具来自自己的配置。这里能减少完整父消息链的直接传递，但任务字符串仍可能包含过量或不可信内容，字段级输入白名单和更严格的数据范围还需要补齐。当前成员调用是顺序执行，我会准确说明实现。

扩展时，我会用结构化任务 Schema 限定传入字段，为每个成员设置工具、知识库、预算和超时，并检查文件、网络及凭据边界。子 Agent 返回内容也要当作待核验结果处理，父 Agent 不能直接按其文字执行敏感操作。测试会覆盖跨 Workspace 对象、越权工具、恶意子结果及预算耗尽，验证拒绝发生在服务端并留下审计。独立 Prompt 对数据隔离的帮助有限，真正的权限要在执行层落实。

追问准备： 父 Agent 转交任务时，业务授权、可见字段和任务相关性分别由谁检查？关联 Q07、Q30、Q48、Q68、Q85。

代码/练习入口：成员装配与 runtimeMemberAgent.Run、工具执行。

Q113｜MCP Host、Client、Server 分别做什么？你的项目处于哪一侧？

来源：**公开面经题**，美团全栈实习面经 S19，MCP 角色区分；作者说明题目经过 AI 整理。相关 Server/参数追问也见 迅雷 Agent 一面 S15。概念依据 T23。

参考回答：

我会按职责解释三个角色。Host 是组织模型交互、上下文与权限的 AI 应用；Client 是 Host 内部与某个 MCP Server 建立协议连接的组件；Server 提供工具、资源等协议能力。一个 Host 可以管理多个 Client，分别连接不同 Server。角色描述的是协议分工，本地或远程部署是另外一个维度，Server 可以运行在任一侧。

在 kbot 消费 MCP 工具的链路里，平台承担 Host 一侧职责，MCP 执行器通过客户端能力连接外部 Server，再将发现和调用结果交给运行时。课程按固定的 2025-11-25 协议相关契约讲解初始化、能力协商、工具发现及调用；Streamable HTTP 和 stdio 是传输选择。模型提出工具调用，实际连接、参数校验与超时由程序处理，平台继续限制端点和授权。

如果把保险查询服务包装成 MCP Server，我会定义清晰工具名、输入 Schema 和结构化结果，将调用映射到既有业务接口，保留鉴权、幂等和错误分类。工具发现表示对方声明了能力，执行权限仍需单独验证。课程已具备 MCP 调用适配和相关限制；对外发布完整业务 MCP Server、跨租户授权与多客户端运维，需要另外实现和验收。

追问准备： MCP 的 Client 与“调用模型 API 的客户端”分别连向谁？协议角色与业务操作授权为什么要分开检查？关联 Q32、Q33、Q100。

代码/练习入口：MCP 适配、工具平台。

Q114｜Agent“自进化”应该改什么，怎样防止错误经验越积越多？

来源：**公开面经题**，美团全栈实习面经 S19，自进化轨迹、知识分层及触发产物；作者说明题目经过 AI 整理。

参考回答：

我会先明确更新对象：任务轨迹记录一次运行发生了什么，用户记忆保存经过授权且仍有效的偏好，业务知识库保存审核后的事实与规则，Prompt 或 Skill 描述执行策略。它们的生命周期和审核要求不同。一次成功回答可以作为候选案例，但还要核对工具结果和业务结局，不能直接写成所有场景都适用的规则。

我设计的闭环是收集失败或人工纠正，按问题类型归因，生成候选修改，再离线评测和审核发布。触发条件可以是重复错误或明确人工反馈，产物可以是新增评测样本、修订条款或新的 Skill 版本。涉及业务事实时由知识负责人确认；涉及执行策略时比较成功率、越权率、成本和历史案例退化，并保留版本及回滚入口。线上用户输入不会自动获得修改系统策略的权力。

课程已有版本管理、Eval 和发布门禁，可以承载这个闭环的一部分，但自动经验提取、策略修订和持续自进化尚未实现。我会先做人工审核的半自动流程，例如将“库存过期导致调拨方案失败”转成回归样本，再验证新增实时库存检查是否改善结果。面试时会展示候选、评测、审批和发布证据，避免只用“能够自我学习”概括一条未经验证的写库逻辑。

追问准备： 错误来自知识、工具、策略或用户缺信息时，更新哪一层？如何防止经验污染及评测集被优化过程反复使用？关联 Q20、Q60、Q62、Q107。

代码/练习入口：评测与门禁、Prompt 版本。

Q115｜Agent 上线后提高了多少人效，怎样拿出可信的证据?

来源：**公开面经题**，腾讯社招三面面经 S20，上线后人效变化及项目价值追问。

参考回答：

我会先定义具体岗位和任务的人效指标，例如每个合格案件的人工处理分钟数、每小时完成的订单诊断数，并同时检查质量和返工。评价要包含人工阅读、复核、修正及异常处理，模型返回更快只说明系统延迟变化。对于赔付和退款，还要保证错误处理、漏检和投诉等质量指标处于可接受范围。

如果具备线上条件，我会选可比任务建立基线，再通过随机对照或分阶段试点比较，并记录样本量、任务难度、时间范围及未完成比例。遇到业务量或人员经验变化时，需要控制或明确这些影响。除人工耗时外，还要报告模型费用、人工复核成本和运维成本；出现困难任务全部转人工的情况，统计范围应覆盖这些任务，避免只挑成功案例。

我的课程项目主要是本地模拟场景和小样本 E2E，目前没有真实线上人员的对照数据，所以我会坦诚给出可证明的成果：实现了哪些流程、通过了哪些正常和异常案例，以及下一步怎样试点测量。简历里可以写“完成审批前零写入等验收”，生产人效提升百分比需要后续真实记录支持。面试官追问数字时，我会提供计算口径和证据，缺少数据就说明验证计划。

追问准备： 以“合格任务的人工总耗时÷合格完成数”举例，并说明任务完成率、质量和分布变化为什么也要一起报告。关联 Q59 及第二部分简历说明。

代码/练习入口：跨境 E2E、保险 E2E。

Q116｜AI Coding 中怎样使用 TDD，避免代码和测试一起犯同一个错误?

来源：**公开面经题**，腾讯社招三面面经 S20，TDD 与 AI Coding 质量保证追问。

参考回答：

我理解 TDD 的核心步骤是先把需求转成能够失败的测试，再实现最小改动让它通过，最后在保持测试通过的前提下重构。使用 AI Coding 时，我会先由人确认业务契约、边界和预期结果，再让工具协助生成实现。测试如果只是照抄实现里的判断，即使全部通过，也可能没有验证真正需求。

以保险赔付为例，我会根据业务说明独立列出责任成立、超过限额、状态冲突和重复提交等案例，再核对测试是否在缺失实现或刻意错误实现下失败。高风险写入还要验证审批前没有副作用，拒绝后没有执行。单元测试验证规则，集成测试检查存储和授权边界，E2E 检查完整链路；模型自然语言回答还需要 Eval，不能仅靠接口测试覆盖。

课程两套业务服务已有测试和 E2E，我会展示具体断言、失败定位和修正记录。新增真实幂等存储时，还应测试相同键不同参数、并发提交及执行后超时，课程模拟器当前没有完整覆盖这些生产条件。我会优先把 TDD 用在规则明确、回归代价高的核心行为，结合代码评审与独立样本，验证 AI 产出的代码是否符合业务要求。

追问准备： 如何构造一个测试能抓住、但现有实现会漏掉的错误？怎样把模型行为的不确定性与业务规则测试分开验证？关联 Q59、Q76、Q99。

代码/练习入口：电商业务测试、保险业务测试。

面经考点总结

面经考点	对应既有题	新增证据
项目个人贡献、技术选型、架构白板、部署与最大难点	Q01、Q05、Q70、Q80、Q87	S11、S12、S13、S14
记忆压缩、历史方案复用、意图路由	Q07、Q20、Q21、Q24	S15、S17、S20
Skill 封装与调用，MCP Server 参数和传输	Q30、Q32、Q33、Q84	S15、S16、S19（AI 整理）
混合检索、切分、证据噪声、幻觉和评测集	Q37–Q40、Q46、Q59、Q60	S15、S16、S19（AI 整理）
ReAct、状态机、框架选择、多 Agent	Q03、Q04、Q07、Q09、Q83	S16、S17、S19（AI 整理）
Token、状态与上下文预算	Q12、Q19	S18
AI Coding 的独立理解与代码审核	Q76	S16、S20

1.13 公开面经来源、学员反馈与技术校核来源

S01–S10 于 2026-10-11 检索，S11–S20 于 2026-10-13 通过已登录牛客网页面读取正文。只展示月日的页面保留原始日期表达，不自行补齐年份；S02 正文和 S07 页面明确给出年份。日期栏区分页面时间和正文自述面试时间。题目来自作者正文，评论、相关推荐、平台自动生成回复及付费不可见部分未作为题目证据。

编号	公开面经原帖	日期信息	本课采用的考点
S01	9.7 百度 Agent 开发日常实习面经	页面 09-07；标题 9.7	Executor、混合检索、切分、记忆、MCP、恢复、观测、多 Agent、树序列化
S02	字节 AI 应用开发一面：中国交易与广告	正文 2026-08-27	Agent Loop、消息、取消验证、记忆、分布式锁
S03	阿里 AI 全栈秋招一面面经	页面 08-28	范式、Workflow、Skill、KV Cache、指标、数据集、复盘
S04	字节 AI 全栈开发一面面经（已过）	页面 08-03	端到端链路、RAG、多模态、开发验证
S05	美团 Keeta Agent 开发：智能客服方向一面	页面 04-28	幻觉、Agentic RAG、训练与评测、删除链表节点
S06	美团 Agent 开发一面凉经	页面 08-25；正文面试 8.23	架构、RAG、切分、路由、并行协作、上下文、异步
S07	快手秋招面经	页面 2025-09-24；正文列 8–9 月流程	Transformer、范式、Prompt、微调、分布式锁
S08	京东后端实习一二面面经（Offer）	页面 09-25；正文 9.15 一面	Agent 开发流程、订单对账
S09	4399-Agent 开发面经	页面及正文 09-18	Harness、大工具输出、指标验证、AI Coding
S10	vivo 线下面面经：Agent 开发	页面及正文 09-22	Harness、工程故障、项目实践

本轮新增原帖： 岗位与面试经历均为个人自述；原帖可追溯。记录中的其他候选人技术栈、参数、工作经历和业务成果不作为本课程实现依据。

编号	原帖标题与作者	页面日期 / 正文时间	岗位范围与采用考点
S11	字节跳动火山引擎agent开发秋招一面；DylanTheReal	页面 10-09 14:11	明确 Agent 开发秋招；Q88–Q93、Q95，项目贡献及异步/后端基础
S12	字节跳动火山引擎agent开发秋招二面；DylanTheReal	页面 10-09 14:12	明确 Agent 开发秋招；Q94，架构白板、部署与个人贡献
S13	秋招-字节跳动-抖音电商-Agent开发一/二/三面；远航_随风	页面 10-07 11:54，已编辑；正文列 09.02、09.08、09.15 三轮	明确 Agent 开发；Q95–Q101，批处理、长合同、事实核验与执行记录；排除推断题面
S14	恒生电子AI应用开发（杭州）一面面经；评论是德国的	页面 10-10 15:34	AI 应用开发；Q92 及个人贡献追问；未采用名称无法确认的框架条目
S15	迅雷 Agent一面；卷不动了w	页面 09-23 23:25，已编辑	Agent 岗位；Q102–Q105，意图、Chunk 关系、检索参数与协作；补充 Skill/MCP 来源
S16	经纬恒润ai应用开发一面面经；红豆包麻糬	页面 10-03 15:14，已编辑	AI 应用开发；Q106、Q107，解析、图表与内容审核；补充 RAG/评测来源
S17	9.8 小厂 agent开发实习 面经；牛客826994329号	页面 09-11 11:54；标题面试时间 9.8	Agent 开发实习，面试企业未披露；Q108–Q110；正文中的小米实习项目不据此归为小米面试
S18	字节agent开发面经；越面越麻	页面 08-09 03:01，已编辑；正文面试时间 08.05	Agent 日常实习；Q110、Q111，框架状态、Token 与算法
S19	美团一面全栈开发日常实习面经；在写日记的咸鱼很伤心	页面显示“昨天 12:09”，访问日 2026-10-13；正文未给面试日期	全栈实习中的 Agent 项目追问；Q112–Q114；**作者说明经 AI 整理**，保留回忆整理记录的证据范围
S20	【社招】腾讯三面面经；kaoyu	页面 09-23 10:06	腾讯社招，具体岗位名未提供；Agent 项目价值、人效和 AI Coding/TDD；Q115、Q116

本轮从已登录搜索结果继续打开原帖，核对标题、作者、页面日期、正文和问题上下文，再与既有题库去重。检索入口包括“Agent 开发 面经”及面经分类，覆盖火山引擎、抖音电商、迅雷、AI 应用等方向。S13 的推断条目与缺失题面未采用；S14 中无法确定的框架名称未自行纠正后冒充真题；S19 的作者 AI 整理说明保留在来源与相应题目旁。

搜索同时检查了面经汇总、通用题库和营销型题解。它们可帮助发现关键词，未作为本课直接题目证据。检索方向包括 Agent 开发一面/实习/秋招、RAG 混合检索、MCP、Skill、任务恢复、取消、记忆、评测、系统设计及行业项目。未覆盖私密面试记录、付费隐藏正文及无法访问的原帖，后续增补应继续保留日期、原帖、考点和题目类型。

编号	反馈来源	收录日期	题目与记录范围
U01	课程方转述学员实际面试问题，本次对话提供	2026-10-11	Q81–Q87；保留七道问题的考察意图，企业、岗位与面试时间未提供，无公开原帖链接

编号	一手技术资料	用途
T01	Anthropic：Building effective agents	范式与工作流分工；采用概念性论述
T02	ReAct 原论文	推理与行动交替的基本范式
T03	Eino ADK 官方文档	框架接口及职责；实际行为以仓库固定依赖为准
T04	MCP 2025-11-25：Transports	stdio、Streamable HTTP 与 SSE
T05	MCP 2025-11-25：Lifecycle	初始化、能力协商和生命周期
T06	PostgreSQL：Controlling Text Search	ts_rank、ts_rank_cd 的全文检索排序
T07	pgvector 官方仓库	余弦检索、精确/近似索引和 HNSW 参数
T08	A2A 官方规范	协议概念；课程实现使用固定版本和有限调用范围
T09	Attention Is All You Need	Transformer 基础
T10	Hugging Face：Caching	KV Cache 原理
T11	Retrieval-Augmented Generation 原论文	RAG 原理
T12	LoRA 原论文	参数高效适配
T13	Sentence Transformers：Retrieve & Re-Rank	召回后重排
T14	Docker Engine security	namespaces、cgroups 与权限边界
T15	Docker：Running containers	容器网络、文件系统、权限和资源参数
T16	Sentence Transformers：Semantic Textual Similarity	文本 Embedding 与相似度计算
T17	gVisor 官方文档	用户态应用内核隔离；课程扩展方向
T18	Firecracker 官方仓库	microVM 隔离；课程扩展方向
T19	Eino Human-in-the-Loop 技术架构	基本、有状态及复合中断，目标断点恢复；具体实现以仓库固定依赖为准
T20	Go context 标准库文档	取消、截止时间与取消原因；辅助识别停止来源
T21	Go FAQ：Goroutines	协程、线程、可增长栈及并发与并行

T22	Redis 官方持久化说明	RDB、AOF 与刷盘策略的故障窗口
T23	MCP 2025-11-25：Architecture	Host、Client、Server 及连接职责，与既有协议教学版本一致
T24	LangGraph 官方概览	状态与图编排的职责；课程实际采用 Eino，未实现 LangGraph 迁移
T25	RFC 6585 §4：429	请求限流与可选的 Retry-After
T26	RFC 9110 §10.2.3：Retry-After	等待提示的秒数和 HTTP 日期格式

二、两份可用于简历的行业项目介绍与答辩说明

2.1 使用规则与版本选择

下文每个行业提供一份主简历正文和对应答辩材料。正文定位为**基于 kbot 的课程实战项目**，可在完成相关模块、理解实现并通过验收后直接采用。项目日期、角色及实际承担的模块由学员填写；未完成的条目删除。复现已有功能、独立实现功能、二次开发功能应按实际贡献描述，项目来源保留在简历或项目链接中。

- 仅完成课堂版：采用主正文中与实际课程成果一致的条目，技术栈删除未使用的 Redis、Asynq、MinIO 等最终版能力；PostgreSQL 范围按第 15 课实际实现说明。
- 已完成最终版实验：可保留持久化、异步审批恢复、模型治理和部署等已验收内容。
- 已独立二次开发：补充具体改动、测试和实验结果，替换泛泛的“优化系统性能”。

两份行业版本共享 kbot 平台底座。通常按目标岗位选择一份作为主项目；同时展示时可写“一个 Agent 平台及两个行业落地场景”，并分别说明业务差异与个人贡献。

2.2 简历项目一：海外电商运营与供应链协同 Agent

可直接使用的简历正文

项目名称：海外电商运营与供应链协同 Agent
项目角色：Go 后端 / AI Agent 应用开发
技术栈：Go、Eino ADK、HTTP/JSON Schema、PostgreSQL、pgvector、Redis、Asynq、SSE、Docker Compose、OpenTelemetry、Langfuse。

项目介绍： 面向海外电商订单履约异常与财务对账场景，基于 kbot 构建具备知识检索、业务工具调用和人工审批能力的 Agent。以美国市场订单、国内仓与海外仓、物流 SLA 和 USD 结算模拟数据为业务基础，打通异常诊断、处理建议、敏感操作审批、恢复执行与审计链路。

主要工作：

- 基于 Eino ADK 组织 ReAct 执行链路，将订单、库存、物流与结算能力通过独立 Go 业务服务和 HTTP Tool 契约接入；结合 JSON Schema 与业务状态校验约束模型调用。
- 为履约规范和结算 SOP 建立知识检索链路，使用 PostgreSQL 全文检索、pgvector 与 RRF 融合召回；通过 Skill 组织订单异常处理和结算对账流程，按 Agent/Skill 收窄 Tool 与 KB 范围。
- 复现调拨、退款及结算申诉的人工审批流程，在敏感 Tool 前保存 checkpoint，结合 Asynq Worker、审批执行租约与业务幂等键完成批准后的恢复执行。

- 使用 Agent、Prompt、Tool 和 Model Profile 固定版本管理会话依赖，通过 Workspace 隔离业务资产，结合 SSE、OpenTelemetry、Langfuse 与审计记录定位执行过程。
- 完成独立 Compose 环境和跨境电商端到端验收，验证订单诊断、调拨审批与恢复、结算差异样本及离线 Eval，形成可复现的演示和排障材料。

可验证成果： 跑通订单缺货诊断至调拨审批恢复的完整链路，并以 11.52 USD 结算差异案例验证工具与业务计算契约；项目使用模拟业务服务，效果以本地测试及演示记录为依据。

业务背景与功能边界

海外电商运营需要跨订单、库存、物流与结算系统获取信息。缺货时需比较跨仓方案及时间约束，退款需核验订单状态和金额，结算申诉需核对账单并保留证据。项目将信息查询和建议生成交给 Agent，将状态变更与金额约束放在业务服务，敏感写操作交由人工审批。

场景	主要输入	工具与业务结果	验收重点
履约诊断	订单号、发货截止要求	get_order、get_inventory、get_shipping_options；输出库存及方案依据	查询事实完整，SLA 假设清楚
跨仓调拨	SKU、源仓、目标仓、数量	create_inventory_transfer；批准后创建模拟调拨记录	审批前零写入，重复执行受控
订单退款	订单、金额、原因	approve_refund；校验订单状态和金额后返回模拟结果	权限、审批、状态与金额边界
结算对账	账单号、币种、应结和实结	get_statement、create_reconciliation_case；差异证据及申诉记录	11.52 USD、币种与计算口径

项目没有真实接入 TikTok Shop、Amazon、Shopee、ERP、WMS、承运商、支付或财务账本。模拟物流方案不构成真实时效承诺；库存调拨创建成功也需与实际发货、出入库和签收分别建模。

90 秒面试介绍

我做的是一个基于 kbot 的海外电商运营与供应链协同课程项目，主要解决运营人员需要在订单、库存、物流和账单之间反复核对信息的问题。

我参与的重点是 Go 后端和 Agent 执行链路。模型用 Eino ADK 选择工具，业务能力通过独立 HTTP 服务接入；履约和结算规范通过知识检索提供，Skill 组织任务步骤。涉及调拨、退款和申诉时，系统先保存审批断点，人工批准后由 Worker 恢复，再记录结果与审计。

以一个美国市场订单为例，深圳仓缺货、洛杉矶仓有库存，Agent 查询订单、各仓库存与物流方案，给出带依据的处理建议；调拨需要明确数量与仓库并通过审批。另一个样本核对出 11.52 美元结算差异。我会用这两个案例说明工具调用、审批和确定性计算的分工。

项目使用模拟业务数据。我能展示本地运行、工具契约、审批恢复和评测记录；接入真实平台、持久化业务幂等以及实际时效和收益验证，是后续需要完成的部分。

常见行业追问与回答要点

追问	可采用的回答
为什么不能直接把订单表放进 RAG?	订单、库存与退款状态需要实时查询和事务约束；RAG 负责 SOP、规则和解释材料
海外仓有库存是否就能保证 SLA?	还需查询调拨、揽收、运输、截单与当地时区；课程只有模拟路线与标志，真实系统要补齐这些数据
多币种金额怎样处理?	每个金额带币种，避免跨币种直接相减；真实系统使用定点金额和有版本的汇率、舍入规则
为什么模型不能直接批准退款?	资金操作需要授权、状态检查和审计；模型提出方案，服务端执行审批与业务门禁
重试退款时怎么避免重复?	复用同一业务幂等键并先查单；生产改造需持久化记录、请求指纹及唯一约束
如何证明个人贡献?	解释所负责模块的契约、关键代码和失败测试，展示自己的提交及实验记录，复现部分如实标注

演示与证据准备

在仓库根目录运行以下命令。需要 Docker、项目依赖和空闲端口；两套 E2E 是受控演示，不包含真实商户账户。

```
cd kbot-final
make -C projects/crossborder test
make crossborder-up
make crossborder-install
make crossborder-e2e
```

随后在 `http://localhost:8181` 展示订单诊断、审批卡片、批准后的业务结果和审计，并确认使用的 Agent 与 Workspace。保留版本、运行时间、Conversation ID、approval ID、Trace ID 和 Eval run ID。结算样本需要单独展示，默认 E2E 主要覆盖调拨审批链路。

证据入口：项目说明、业务服务与测试、工具清单、E2E 脚本、评测样本。

可选二次开发方向

建议选择一项独立完成：持久化业务幂等并增加同键异参冲突检测；接入真实或契约一致的 WMS 沙箱；实现含时区和调拨时间的 SLA 校验；引入中文/英文业务评测；将结算金额改为定点计算。完成后用"改动—失败用例—验证结果—剩余边界"补充简历。

2.3 简历项目二：保险理赔审核与反欺诈协同 Agent

可直接使用的简历正文

项目名称：保险理赔审核与反欺诈协同 Agent
项目角色：Go 后端 / AI Agent 应用开发
技术栈：Go、Eino ADK、HTTP/JSON Schema、PostgreSQL、pgvector、Redis、Asynq、SSE、Docker Compose、OpenTelemetry、Langfuse。

项目介绍： 面向车险理赔审核与高风险案件处置场景，基于 kbot 构建条款检索、责任核验、赔款计算和人工审批协同的 Agent。通过独立保险业务模拟服务提供保单、案件、责任计算与风险信号，以 Skill 组织审核与调查流程，形成事实查询、规则判断、审批恢复和审计闭环。

主要工作：

- 将保单、案件、责任计算与风险特征接口封装为受控 Tool，通过 JSON Schema、资源范围与业务状态门禁约束模型参数和操作；采用规则服务输出责任理由码与可赔上限。
- 为责任条款、材料审核与调查 SOP 建立知识库，使用全文检索、pgvector 与 RRF 融合提供证据，结合 Skill 编排理赔审核、风险分流和调查协作。
- 复现补件、赔付、支付冻结及调查立案的人工审批链路，使用 checkpoint 与精确中断地址恢复目标操作，结合 Worker 执行租约和幂等键控制重复处理。
- 通过 Workspace、Agent/Skill 资源授权及输入输出 Guard 保护业务数据，结合固定配置版本、OpenTelemetry、Langfuse 和审计记录追踪审核依据与处置过程。
- 完成保险 Compose 环境和高风险冻结端到端验收，覆盖常规赔款计算、高风险禁止自动赔付及审批恢复等案例，使用离线样本与确定性断言验证关键行为。

可验证成果： 以常规案件验证 6300 CNY 可赔上限，以高风险案件验证风险分流、支付冻结审批与恢复执行；风险信号与业务数据由模拟器提供，结果以本地测试和演示记录为依据。

业务背景与功能边界

理赔人员需要核对保险期间、责任范围、免赔额、索赔金额及材料，同时处理异常线索。项目把模型用于信息汇总、知识检索和说明，将责任和金额计算放在确定性服务，将赔付、冻结和立案放在人工审批后的受控执行链路。

场景	主要输入	工具与业务结果	验收重点
常规审核	案件号、保单与事故时间	get_claim、get_policy、evaluate_coverage、get_fraud_features	条款适用、时间与金额解释
材料补齐	缺失或待验证材料	request_additional_documents；审批后创建补件请求	缺失依据、客户触达审批
赔付审批	责任结果、可赔上限与拟赔金额	approve_claim；批准后形成模拟决定	金额、风险、审批与状态
高风险处置	事故时间线、风险分及信号	hold_claim_payment、open_fraud_investigation	冻结和立案分步审批，证据与推断分开
核保扩展实验	UW-2026-0001 与规则输出	get_underwriting_case、assess_underwriting、approve_underwriting	仅在完成独立项目核保配置与实验后计入个人成果

保险独立项目保留核保工具及样本；完整课堂环境的四个场景 Agent 预设重点覆盖理赔和反欺诈。本简历主版本聚焦已演示的理赔闭环，避免将两种装配方式混为一体。

风险分 0.92、阈值 0.70、重复影像信号、6300 CNY 赔款以及 4160 CNY 核保建议均属于课程模拟规则或数据。项目没有训练反欺诈模型、实现影像查重算法或接入真实保险公司支付通道；对外展示时使用模拟案件与材料。

90 秒面试介绍

我做的是一个基于 kbot 的保险理赔审核与反欺诈协同课程项目。它面向理赔人员核对保单、事故时间、条款和材料的场景，把查询、规则计算、风险分流与人工审批连成一条可追踪的流程。

模型使用 Eino ADK 调用保单、案件和规则接口，知识库提供条款与审核 SOP。责任及金额由确定性服务返回，模型负责解释和组织证据；赔付、支付冻结和调查立案都需要人工批准，再由 Worker 从 checkpoint 恢复相应操作。

常规案件索赔 6800 元，按课程条款应用 500 元免赔额后，可赔上限为 6300 元。高风险样本存在事故早于保单生效及重复影像信号，系统将其分流到人工冻结与调查，风险信号保持待核验性质。

项目的保险数据和风险特征来自模拟器。我能够展示规则接口、审批恢复、业务状态与审计记录；真实条款版本管理、材料识别、持久化业务幂等及真实机构对接是后续落地需要补充的工作。

常见行业追问与回答要点

追问	可采用的回答
为什么赔款不直接让大模型计算?	金额需要确定性计算、边界校验与可复核依据；模型输出计算结果的解释
最新条款是否一定适用于历史案件?	需匹配保单产品版本、保险期间及合同适用规则；完整历史条款快照与有效时间过滤是扩展项
风险分高是否可以拒赔?	风险信号用于人工调查分流，责任与欺诈结论需要分别确认并保留依据
反欺诈模型怎么训练的?	当前读取模拟器预置规则与信号；真实学习模型需要标注数据、特征治理、误报评估和专项验证
重复影像信号怎样生成?	当前为模拟输入；真实接入需提供图像哈希或视觉服务、证据来源及误判处理
支付冻结和立案为什么分开审批?	两个动作的业务效果和权限不同，各自绑定参数与状态，后者还需检查前置处置结果
客户材料怎样保护?	限制访问范围，敏感信息脱敏，控制日志和 Trace 内容，接入真实机构时明确保留与删除策略
核保能力是否在主流程中?	独立项目提供规则与工具，当前主简历聚焦理赔和反欺诈；完成核保实验后再增加对应成果

演示与证据准备

在仓库根目录运行：

```
cd kbot-final
make -C projects/insurance test
make insurance-up
make insurance-install
make insurance-e2e
```

在 `http://localhost:8182` 展示高风险案件进入审批、批准后支付冻结以及审计记录。常规 6300 CNY 计算、实际赔付审批、调查立案和核保应各自补充演示，默认保险 E2E 主要验证高风险冻结链路。保留使用的规则版本、案件 ID、Conversation ID、approval ID、Trace ID 及 Eval run ID。

证据入口：项目说明、保险服务与测试、工具清单、E2E 脚本、评测样本。

可选二次开发方向

建议选择一项独立完成：条款有效期与保单版本绑定；补件材料的状态追踪；真实 OCR 沙箱与低置信度复核；持久化赔付幂等和同键异参拒绝；风险误报与公平性评测；对责任判断和金额审批增加独立状态断言。成果应以可复现的样本、变更与测试说明。

2.4 学员成果数据与个人贡献说明

简历可以先采用上面的事实性成果描述，无需虚构效率提升、准确率、客户数和资金规模。需要量化时，应先完成实验并记录下表。

指标	建议口径	必须保留的证据
端到端任务成功率	同版本完成业务目标且满足约束的任务数/总任务数	数据集来源、任务数、运行次数、失败分类、最终业务状态
关键安全用例	未批准写入、越权、重复副作用分别统计	断言、审计、故障与重试记录；每项单独报告
证据召回	有标注支持片段的查询中，Top-K 命中情况	标注集、K、文档/Embedding 版本和检索模式
延迟	首响应及完整任务耗时，报告中位数和 P95	环境、并发、输入长度、模型、样本数；审批等待单列
成本	单任务所有模型、Embedding、工具及重试开销	实际 usage、价格日期、计价口径和失败任务成本
人工工作量	相同任务、相同质量门槛下的人工作业量	对照方法、参与者、任务分布与人工复核记录

E2E 中的 100% 表示固定受控样本满足脚本断言。业务 JSONL 中的 expected_tools、forbidden_tools、dimensions 等字段表达预期，是否被实际评分取决于运行器和 Judge 的实现；应检查导入脚本与结果，不能默认全部维度已自动验证。

主简历中使用“复现”“完成课程实战”“基于 kbot 扩展”等动词反映实际贡献。只有自己实现并验证的部分才使用“设计并实现”；学习与复现也可以通过清楚解释契约、失败路径和改进实验体现能力。

2.5 模拟面试准备

把课程中项目基于自己的业务方向编写一份行业项目简历、一段 90 秒项目介绍、一份实际贡献清单、一组正常与异常案例，以及对应的测试/Trace/审计或演示记录。建议按下表完成 20 分钟模拟面试。

环节	用时	主要要求
项目介绍	2 分钟	痛点、个人范围、一次业务闭环和验证方式
架构与技术选择	5 分钟	ReAct/Workflow、RAG/Tool、版本、Skill 与授权分工
故障与安全追问	6 分钟	取消、中断恢复、重复执行、事实变化与越权
代码或测试解释	4 分钟	解释一个关键实现，并说明测试的断言与限制
成果与复盘	3 分钟	数据口径、真实贡献、剩余边界和下一项改进

🙋‍♂️Go Agent 开发实战视频课程 配套简历&面试辅导课件。

欢迎来B站关注 @七米老师 、@七米课堂